MG Domo - version 2.6.35 du 17/10/2026
Plugin logiciel (aucun matériel requis)
Écosystème MG Domo : présentation, installation et liste des plugins dans MG_Domo.pdf

Plugin mgDomo

Présentation

MG Domo est le socle de l'écosystème des plugins **mg***. Il porte les **utilisateurs** de la maison (avec en option leur téléphone équipé de l'application mgDomo, Android), la **bibliothèque mg** de fonctions pour vos scénarios et vos commandes, les **widgets** de dashboard, l'**installation et la mise à jour** des plugins mg*, le **menu** du navigateur, le **réseau privé** Tailscale et les **demandes d'aide**.

La page MG Domo (menu Plugins > MG Domo) se compose d'une zone **Gestion** (Configuration, Santé, Exemples, Community) et de dix onglets, dans l'ordre : Plugins, Utilisateurs, Bibliothèques, Variables mgDomo, Tables SQL, Widgets, Menu MG Domo, Réseau privé, Demande d'intervention, VM/CT Proxmox. Chaque onglet s'ouvre sur un encart qui explique son rôle.

Utilisateurs

Les personnes de la maison sont des équipements du socle : l'onglet **Utilisateurs** de la page MG Domo les liste à gauche ; cliquez sur une ligne pour ouvrir la fiche de la personne à droite. Le bouton vert **Ajouter** crée un utilisateur (l'identifiant est demandé).

La fiche porte l'**identité** de la personne, écrite à chaque sauvegarde dans l'annuaire du socle (table `_tabUsers`), que lisent les messages de la maison (`mg::message('SMS:@U1', ...)`, `MAIL:@U2`), l'alarme de présence et les plugins :

- **Nom de l'équipement = identifiant** : code court et unique (U1, U2...), clé de tout le reste : destinataire `@nom`, équipement mgGeofence et mgGarmin du même nom, `mg::apk`, paramètre `equip/passerelle_APK`. Lettres, chiffres, `_` ou `-`, 20 caractères au plus ; 1 à 3 pour une personne géolocalisée (clé des positions et des activités). **Définitif** après la première sauvegarde : pour le changer, supprimez la fiche et recréez-la.
- **Nom complet**, **Numéro de téléphone**, **Adresse mail**.
- **Membre de la maison** : coché, la personne est inscrite dans l'annuaire ; décoché, c'est un utilisateur *technique* (une passerelle SMS fixe, par exemple), jamais destinataire ni évalué par la présence. Sans photo, un utilisateur technique se reconnaît à son icône (carte SIM pour une passerelle SMS, puce sinon) là où une personne a une silhouette.
- **Habite la maison** : coché, sa présence compte (alarme de présence) ; décoché, personne de l'entourage, joignable mais jamais attendue.
- **Utilisateur principal** : coché, cette personne reçoit les SMS et mails que les plugins envoient sans viser quelqu'un en particulier (alertes de coupure, d'équipement

injoignable…). Un seul utilisateur principal : en cocher un décoche les autres. Aucun coché : le premier utilisateur qui habite la maison et a une adresse mail.
- **Photo** et **Couleur** : l'identité visuelle reprise par les plugins (accueil et carte de mgGeofence, graphiques). Le bouton **Parcourir…** de la fiche envoie l'image choisie sur le serveur (`plugins/mgDomo/data/users/`, réservé aux réseaux privés) ; on peut aussi saisir le chemin d'une image déjà présente.

Un annuaire rempli autrefois à la main (mgTabulator) n'est pas perdu : l'onglet signale les lignes sans fiche et crée la fiche en un clic.

Option : téléphone connecté (application mgDomo)

L'application mgDomo, installée sur un téléphone Android, relie le téléphone à Jeedom : elle envoie ses positions GPS et ses pas (suivi par le plugin mgGeofence), l'état de l'appareil (batterie, réseau, volume, sonnerie), et exécute les ordres de la maison : annonce vocale, notification, réglage du volume, sonnerie pour retrouver le téléphone, lampe, vibreur, envoi de SMS depuis la carte SIM, pilotage des enceintes Bluetooth (choix, connexion, radio, maintien en éveil).

Cocher **Téléphone connecté** dans la fiche d'un utilisateur fait apparaître :

- **Capacités** : Suivi GPS (téléphone porté, géolocalisation), Passerelle SMS (téléphone avec carte SIM), Passerelle mail (téléphone dont le compte Google envoie les mails de la maison, commande « Envoyer un mail »), Enceintes Bluetooth (téléphone fixe). Chaque capacité crée ses commandes à la sauvegarde.
- **Appairage** : sauvegardez la fiche, puis scannez son QR code avec l'application (bouton « Scanner le QR code de configuration »). Le QR code porte l'adresse de Jeedom, le jeton propre au téléphone et l'identifiant de l'équipement. La pastille du titre indique la version de l'application installée et si une mise à jour est disponible : l'application se met à jour seule (téléchargement depuis ce serveur). Un téléphone resté sur une ancienne version est aussi cité dans le signalement quotidien des mises à jour (voir **Plugins**).
- **Widget** : « Widget mgDomo » présente le téléphone en une tuile compacte (état, messages, retrouver le téléphone, enceinte, maintien en éveil, maintenance) ; seules les commandes cochées « Afficher » y apparaissent.
- **Surveillance** : un téléphone muet au-delà de quatre signaux de vie déclenche un réveil puis une alerte (jamais par SMS, le canal en panne) ; la reprise de contact est signalée.
- **Réglages de l'application** : **Signal de vie** (minutes entre deux contacts de fond, 15 par défaut), **Journal de l'application** (Aucun, Essentiel ou Détaillé : traces envoyées au serveur, log `mgDomo_apk_<identifiant>`), **Bouton Homepage** (un bouton orange en tête de l'écran de l'application, avec son adresse et son libellé). Le **modèle du téléphone** et son **adresse Tailscale** s'affichent en lecture seule dès que l'application s'est annoncée.
- **Commandes créées** : informations Volume, Batterie, En charge, Mode sonnerie, Ne pas déranger, Réseau, Dernier contact, Version APK ; actions Notifier, Lire un texte (TTS), Régler le volume, Sonner, Arrêter la sonnerie, Flash lampe, Vibrer, Relancer Tailscale, Réveil prioritaire (FCM), Redémarrer le service, Arrêt complet ; avec le suivi GPS : Forcer la position, Vider la file ; passerelle SMS : Envoyer un SMS ; passerelle mail : Envoyer un mail ; enceintes Bluetooth : informations Enceinte active, Enceintes, Lecture en cours,

Maintien en éveil, Enceinte choisie (trois informations techniques sont créées masquées) et actions Choisir une enceinte, Enceinte marche/arrêt, Lire une radio, Relancer la radio, Lire une adresse, Arrêter la lecture, Maintien en éveil marche/arrêt, Force du pic, Durée du pic, Intervalle entre pics, Force du fond.

Dans la configuration du plugin (Plugins > Gestion des plugins > MG Domo), les groupes suivants :

- **Widgets** : **Widgets actifs** (le style et le code des widgets mgDomo sont recopiés dans un bloc réservé de `custom.css` et `custom.js`, puis tenus à jour toutes les 5 minutes) et **Recopier custom.css / js** (recopie immédiate).
- **Classe mg** : **Accessible partout** (la bibliothèque mg dans les formules et les scénarios, sans include). Le bouton **Aide sur la classe mg**, à droite de la case, ouvre la liste des fonctions de la bibliothèque, classées par thème (messages, alertes, commandes, variables, scénarios, réglages, dates...), avec pour chacune ses paramètres, son retour et un exemple d'appel. Elle figure aussi en annexe de la documentation PDF du plugin.
- **Application téléphone** : le jeton de l'application (à régénérer en cas de doute, puis scanner de nouveau les QR codes : chaque téléphone reçoit dans son QR code un jeton qui lui est propre, révocable depuis sa fiche), le Wi-Fi du domicile (adresse locale de Jeedom essayée en premier) et la **rétention des positions** (jours, 365 par défaut, 0 = aucune purge : chaque nuit, les positions plus anciennes sont supprimées ; un jour supprimé ne peut plus être recalculé par mgGeofence).
- **Accès Internet** : **Surveillance** Désactivée, Observation (la sonde tourne, chaque décision est seulement écrite dans le log mgDomo) ou Active (sonde chaque minute ; coupure confirmée en une minute environ : alerte par SMS, commande mémo à 1, drapeau lu par le PC pour le secours 4G).
- **Classement des plugins mg* dans le menu** : chaque plugin sous sa catégorie d'origine, ou tous regroupés sous MG Domo (bouton **Appliquer**, menu mis à jour au rechargement de la page).

Le groupe **Machine de référence** n'apparaît que si le code source non minifié du socle est présent ; le groupe **Jeton GitHub** seulement sur la machine déclarée machine de référence (celle qui construit la distribution).

Exemples de widgets

Sur une station neuve, l'installation crée un équipement de démonstration, **Démo widgets**, rangé dans l'objet **Exemples MG Domo** du dashboard. Il présente un exemplaire de chaque modèle de widget livré avec le plugin : thermomètre, hygromètre, baromètre, boussole, jauges, vumètre, manche à air, compteur, badge, volets, fenêtre, porte, store, présence, voyants, texte, boutons, lampes et curseurs.

- Les boutons, lampes et curseurs **réagissent au clic** : ils pilotent une valeur interne à l'équipement, sans aucun matériel. Rien d'autre n'est commandé dans votre installation.
- Pour voir comment un widget est réglé : ouvrez l'équipement (plugin **Virtuel**), puis la roue dentée d'une commande, onglet **Affichage** : le widget choisi et ses paramètres (`ty`

`pe`, `imgName`, `titre`…) s'y lisent. Recopiez-les sur vos propres commandes, ou passez par l'onglet **Widgets** de la page MG Domo.

- La carte **Exemples** de la page MG Domo (zone Gestion) **supprime** cet équipement quand vous n'en avez plus besoin, ou le **recrée** à la demande. Seul cet équipement est concerné ; l'objet n'est retiré que s'il est vide.
- L'équipement n'est jamais créé sur une station qui utilise déjà des widgets MG Domo. Une fois supprimé par la carte **Exemples**, il n'est jamais recréé d'office, ni par une mise à jour du plugin, ni par sa réactivation.
- Trois commandes (**Adresse IP via class mg**, **Nom de la station**, **Durée formatée**) montrent comment appeler une fonction de la classe `mg` dans le **calcul** d'une commande virtuelle, par exemple `getConfigJeedom(core,internalAddr)`. Règles d'écriture : le nom de la fonction **sans** `mg::`, les arguments **sans guillemets**. Ces formules demandent l'option **Accessible partout** (groupe « Classe mg » de la configuration du plugin) (active par défaut sur une station neuve) ; vos propres fonctions utilisateur (`data/php/user.function.class.php`) sont conservées, un bloc repéré y est simplement ajouté.
- L'objet **Exemples MG Domo** est rangé sous l'objet racine de la station : il apparaît donc directement sur le dashboard, sans rien sélectionner.

  Widgets affichés en valeurs brutes, sans image ? Les widgets mgDomo ont besoin de la *personnalisation avancée* de Jeedom (Réglages > Système > Configuration, onglet **Interface**, section **Personnalisation**, ligne **Avancée**). Le plugin l'active lui-même à l'installation ; si elle a été décochée depuis, le contrôle **Santé** de la page MG Domo le signale. Après l'avoir recochée, rechargez la page (Ctrl+Maj+R).

Contrôle de santé

La carte **Santé** de la zone Gestion ouvre le rapport d'anomalies de l'écosystème. Contrôles effectués : bibliothèque mg et widgets en place, tables attendues présentes, installation des plugins mg*, variables mgDomo définies, commandes partagées qui existent, plugins actifs sans aucun équipement, plugins liés requis (Virtuel, Mail), compte de téléintervention oublié.

- Chaque **anomalie** est rangée par catégorie et accompagnée de la colonne **Que faire** : le geste à accomplir.
- Les **avertissements de robustesse** signalent ce qui fonctionne mais mérite attention.
- Quand tout est en ordre, le rapport affiche « Aucune anomalie » et le nombre de contrôles passés.
- Le bouton **Relancer le contrôle** rejoue le rapport après une correction.

Exemples de contrôles : « Bibliothèque mg inaccessible alors que "Accessible partout" est coché » (décochez puis recochez l'option dans la configuration de MG Domo, sinon les scénarios ne peuvent pas appeler la bibliothèque) ; « Personnalisation avancée de Jeedom désactivée » (le style et le code des widgets mgDomo ne sont pas chargés) ; « Compte administrateur de téléintervention actif » (à révoquer dès la fin de l'intervention).

La liste **Reste à faire pour terminer cette installation**, en tête de l'onglet Plugins, complète ce contrôle : envoi des mails (équipement du plugin Mail à renseigner dans `_tabPara`

`ms`), localisation de la maison, matériel déclaré présent sans équipement, points à compléter dans la fiche d'un utilisateur, paramètres à renseigner selon les fonctions utilisées. Chaque ligne porte un lien **Ouvrir** vers la page concernée.

Variables mgDomo

Les plugins mgDomo lisent quelques **variables Jeedom** (Outils > Variables) pour connaître le contexte de la maison : `nePasDeranger`, `heureReveil`, `NuitSalon`, `NuitExt`, `Alarme`... L'onglet **Variables mgDomo** de la page MG Domo les liste toutes, avec leur valeur, leur description et les plugins qui s'en servent.

- **En rouge « non définie »** : variable que **vos scénarios** doivent produire (par exemple un scénario qui met `NuitExt` à 1 au coucher du soleil). Tant qu'elle n'existe pas, le plugin concerné utilise une valeur de repos.
- **En gris « pas encore écrite »** : variable écrite par un plugin lui-même (`saison`) : il n'y a rien à faire.
- Sur une station neuve, l'installation crée les variables attendues à leur valeur de repos (0). Une variable qui existe déjà n'est jamais modifiée.
- Le bouton bulle d'une ligne permet de saisir **votre** description ; elle remplace celle fournie par le plugin et n'est pas écrasée par les mises à jour.
- Le bouton **Modifier la valeur** d'une ligne change la valeur de la variable sans passer par Outils > Variables.

Plugins

L'onglet **Plugins** liste les plugins mg* installés sur cette station, leur état et leur version, en trois tableaux :

- **Plugins logiciels** : ceux qui ne dépendent d'aucun appareil particulier (socle, outils, automatismes, services).
- **Plugins liés à un matériel** : la colonne **Matériel requis** nomme l'appareil. Tant que la case **Présent** n'est pas cochée, le plugin attend son matériel : aucun équipement n'est demandé et le contrôle de santé ne le signale pas. Cochez la case quand l'appareil est installé et joignable par Jeedom : le plugin doit alors avoir un équipement.
- **Plugins liés à l'écosystème mgDomo** : les plugins du market dont le socle a besoin (Virtuel, Mail), leur rôle et leurs boutons **Installer** / **Activer**.

Après le nom, la colonne **Création** donne la date de création du plugin, ou du script dont il est issu. Sur la machine qui publie la distribution, la case **Ne pas publier** retire un plugin de la distribution : au prochain **Make master**, son archive et sa documentation sont supprimées du dépôt en ligne. Sur les autres installations, le plugin est alors signalé « Retiré de la distribution » et la **Mise à jour de la sélection** suivante le supprime, avec ses équipements et leur historique (la confirmation le nomme). Décochée, le plugin est republié au Make master suivant et peut être réinstallé. Un plugin absent de la distribution garde l'affichage de sa version installée, en gris, sans mise à jour proposée.

Pour chaque plugin : le nombre d'**équipements**, les **actions** (Installer, Activer, Désactiver ; désactiver un plugin arrête tous ses équipements) et les **outils** (bouton **Changelog**, qui ouvre les nouveautés du plugin).

Le **bouton de version** (icône de nuage suivie du numéro de version) compare la version installée à la version publiée dans la distribution en ligne de l'écosystème :

- vert, la version installée : identique à la version publiée (« A jour ») ; un clic la retélécharge quand même ;
- orange, « version installée → version publiée » : une mise à jour est disponible ; un clic la télécharge et l'installe, **intégrité vérifiée** (les fichiers du plugin installé sont remplacés) ;
- bouton grisé, « Plugin en version bêta » : installation et mise à jour refusées sur les autres installations tant que le plugin n'est pas libéré par son auteur.

Mise à jour groupée : chaque plugin qui a une mise à jour disponible porte une **case à gauche de sa ligne**, cochée d'office. Décochez celles que vous voulez écarter (la case **Tout cocher** coche ou décoche tout), puis cliquez sur **Mettre à jour la sélection** : les plugins cochés sont mis à jour l'un après l'autre, MG Domo en premier, avec le même contrôle d'intégrité que le bouton de version. Un échec n'empêche pas les suivants ; un bilan s'affiche à la fin et la liste se recharge.

Le bouton **Rafraîchir** relit la liste. Sur la machine qui construit la distribution, la carte **Make master** de la zone Gestion publie les versions ; elle n'apparaît pas ailleurs.

Signalement quotidien des mises à jour

Chaque matin à partir de 7 h, une seule fois par jour, MG Domo vérifie s'il existe une version plus récente :

- des plugins mg* installés (comparés à la distribution en ligne) ;
- de l'extension du menu, d'après la version que l'extension a annoncée lors de sa dernière lecture du menu ;
- de l'application mgDomo de chaque téléphone.

S'il y en a, un message unique les regroupe, par exemple « Mises à jour disponibles : mgVolets 2.1.69 → 2.1.70, application de U1 0.7.11 → 0.7.12 », l'application étant désignée par l'identifiant de l'utilisateur. Il apparaît dans le **centre de messages** de Jeedom (il remplace celui de la veille) et dans la **timeline** mgDomo (log `_Timeline`). Quand tout est à jour, rien n'est écrit et le message de la veille disparaît. Sur la machine qui construit la distribution, les plugins ne sont jamais signalés : elle est la référence.

Bibliothèques et outils externes

L'onglet **Bibliothèques** inventorie les composants tiers embarqués par MG Domo (dossier `3rdparty/`) au service de la bibliothèque mg : agenda iCalendar (`mg::getICS()`), calcul du soleil et de la lune (`mg::getSoleil()`), synthèse vocale ecrireVoiceRSS (`mg::VoiceRSS()`), génération des QR codes de l'application, et quelques composants conservés sans être appelés. Pour

chacun : version, usage, source, taille, dernière mise à jour, et un **statut** qui distingue ce qui est appelé aujourd'hui (actif) de ce qui est conservé sans être appelé (dormant).

La section **Outils externes** du même onglet donne accès aux outils interactifs servis dans une fenêtre ou par une adresse directe : **Adminer** (administration de la base de données) et **phpSysInfo** (informations système du serveur). Ces deux outils ne sont pas livrés avec le plugin : le bouton **Installer** de leur ligne les télécharge depuis la distribution MG Domo (contrôle d'intégrité compris), **Désinstaller** les retire. Une installation qui les avait déjà les garde. L'extension navigateur **MG Domo - Menu** y figure aussi, avec sa version.

Tables SQL

L'onglet **Tables SQL** liste les tables de l'écosystème (préfixe `_tab`) : nom, nombre de lignes, taille, et les plugins qui les **utilisent**. Leur visualisation et leur édition sont confiées au plugin **MG Tabulator** : le bouton **Ouvrir** y ouvre la table choisie. C'est là que se renseignent les paramètres du socle (table `_tabParams`, colonne remarque), par exemple l'équipement du plugin Mail qui envoie les messages.

Médias des enceintes (partage « musicshare »)

Les jingles et l'annonce vocale du Sonos sont des fichiers mp3 rangés dans le **dossier des médias** : par défaut `plugins/mgDomo/data/media`, livré avec les jingles. Le paramètre **Media / PathMedia** de la table `_tabParams` permet d'en choisir un autre (chemin absolu d'un dossier qui existe déjà, ou chemin relatif à la racine de Jeedom).

Le Sonos lit ces fichiers par un **partage réseau Samba** nommé `musicshare`. À l'installation de MG Domo :

- si un partage `musicshare` existe déjà, il n'est **jamais modifié** : son dossier devient le dossier des médias et les jingles livrés y sont copiés s'ils manquent ;
- sinon, **Samba est installé** s'il manque et le partage est créé sur le dossier des médias. Le fichier de configuration d'origine est sauvegardé à côté (`smb.conf.mgdomo.<date>`), les réglages généraux de Samba ne sont pas touchés. Un message du centre de messages Jeedom l'annonce.

Le partage est ouvert **sans mot de passe, en lecture et en écriture**, à tous les appareils de votre réseau local : c'est ce qu'attendent le Sonos et le plugin sonos3, qui y dépose ses propres annonces (sous-dossier `tts`). Comme ce dossier est dans celui du serveur web, deux protections empêchent qu'un fichier déposé par le partage agisse sur Jeedom :

- le dossier par défaut `plugins/mgDomo/data/media` est **fermé au web** : aucun de ses fichiers n'est servi par Jeedom (le Sonos les lit par le partage) ;
- le partage **refuse** les fichiers cachés (nom commençant par un point) et les noms de programmes PHP (`.php`, `.phtml`, `.phar`).

Un partage créé par MG Domo avant la version 2.5.69 reçoit la seconde protection dans les 5 minutes qui suivent la mise à jour ; si elle ne peut pas être posée (droits, configuration Samba refusée), une erreur est inscrite au journal mgDomo et l'essai est refait le lendemain. Un

partage que vous aviez créé vous-même n'est pas modifié : s'il est ouvert sans mot de passe en écriture, ajoutez-y vous-même la ligne `veto files = /.*/*.php*/*.phtml*/*.phar*/`. Un dossier des médias choisi ailleurs dans le dossier du serveur web n'est pas fermé au web par MG Domo.

Il reste à faire dans l'application Sonos : ajouter le partage à la bibliothèque musicale, puis créer les favoris **VoiceRSS.mp3** et **jingle_01.mp3**. Tant que le partage manque ou que Samba est arrêté alors qu'un Sonos est déclaré (paramètre `equip / equipSonos`), la liste **Reste à faire** de l'onglet Plugins affiche le point « Partage des médias Sonos ».

Widgets

L'onglet **Widgets** gère les **presets** d'affichage des widgets mgDomo, c'est-à-dire le modèle appliqué aux commandes. Un preset est enregistré dans la table `_tabWidgets`. La documentation du framework décrit les types disponibles, leurs paramètres et des exemples, à lire avant la première personnalisation.

- **Réglages globaux** : titre et minuterie des widgets (couleurs de texte et de fond), **couleurs des catégories** (barre de titre et cadre des équipements selon leur première catégorie cochée, widgets de scénario, widgets sans catégorie ; vide = couleur du thème), opacité et texte des titres (thème ou contraste automatique), épaisseur du cadre des widgets. Les boutons **Enregistrer** et **Rétablir les défauts** s'appliquent à chaque groupe.
- **Type de widget** : choisissez un type dans le bandeau pour ouvrir sa **personnalisation** : preset Défaut, Ajouter, Enregistrer, Supprimer, commandes liées, ajout d'un paramètre.
- **Prévisualisation** : choisissez une commande (filtre : toutes les commandes du type), regardez le rendu, puis **Appliquer à la commande**.

Documentation détaillée

La documentation de référence du framework est organisée dans les documents suivants :

- Framework de widgets mgDomo — référence complète des widgets de dashboard : types disponibles, paramètres, exemples.
- Générateur de tableaux (tableGenerator) — création et mise en forme de tableaux dynamiques.

Menu MG Domo

Le menu MG Domo est une barre de boutons affichée en haut de chaque page Jeedom et des sites que vous y ajoutez. Chaque bouton ouvre un design, une vue, une page de Jeedom, la page d'un plugin ou un site, dans son propre onglet ; tous les onglets du menu sont rangés dans le groupe d'onglets **Jeedom** du navigateur. Un clic sur un bouton active son onglet s'il existe (dans la fenêtre du menu, ou dans toutes les fenêtres si vous l'avez réglé ainsi), sinon le crée. En fonctionnement **Onglets**, au démarrage du navigateur, les onglets manquants

sont rouverts et mis en sommeil ; en fonctionnement **Classique** (par défaut), un seul onglet sert au menu.

La barre est affichée par l'extension navigateur **MG Domo - Menu** (Edge et Chrome sur PC). Le menu lui-même (boutons, couleurs, icônes) se règle dans l'onglet **Menu MG Domo** de la page MG Domo.

Installer l'extension

Une fois par PC :

1. Dans l'onglet **Menu MG Domo**, cliquez sur **Télécharger le zip** et dézippez-le dans un dossier local, par exemple `C:\mgDomo\extension_menu` (un dossier réseau ne convient pas : le navigateur relit le dossier à chaque démarrage).
2. Ouvrez `edge://extensions` (Chrome : `chrome://extensions`), activez le **Mode développeur**, cliquez sur **Charger l'extension non empaquetée** et choisissez ce dossier.
3. Ouvrez une page Jeedom, puis cliquez sur l'icône de l'extension : sous « Nouvelle adresse Jeedom ? », cliquez sur **adopter**. L'extension lit alors le menu. Une adresse n'est jamais retenue sans cet accord : une page qui imiterait Jeedom ne peut pas remplacer vos boutons.
4. Cliquez sur l'icône de l'extension (pièce de puzzle dans la barre d'Edge, épinglez-la si vous voulez), puis le bouton **Autoriser** (section « Sites à autoriser ») : le navigateur demande une fois l'accès à Jeedom et aux sites externes du menu.

L'onglet **Menu MG Domo** affiche la version disponible sur ce Jeedom et la version installée dans le navigateur courant. Mise à jour : dézippez la nouvelle version par-dessus le dossier local, puis cliquez sur **Recharger l'extension** (bouton de l'onglet, extension 2.1.33 ou plus : elle se recharge elle-même) ; à défaut, **Recharger** dans `edge://extensions`. L'extension annonce sa version à Jeedom chaque fois qu'elle lit le menu : une extension en retard est citée dans le signalement quotidien des mises à jour (voir **Plugins**).

La fenêtre de l'icône montre l'état du menu et propose **Relire le menu** (après une modification) ; selon le fonctionnement, **Rétablir les onglets** (recrée le groupe et les onglets manquants, puis met au premier plan le bouton « Au démarrage ») ou l'onglet du menu avec **Prendre cet onglet comme onglet du menu** ; et, seulement s'il en reste, les sites à autoriser. Un repli **Diagnostic** donne les adresses Jeedom connues, le journal et deux outils (remise en service de la barre, vérification des onglets). Détails et limites dans le README du zip.

Sans réglage de votre part, le menu propose Dashboard, Scénarios, Logs, MG Domo, Plugin Virtuel, MG Tabulator et MG Météo France (les trois plugins que MG Domo installe d'office).

Modifier le menu

Tout se règle dans l'onglet **Menu MG Domo**, sans écrire de fichier. Chaque ligne du tableau est un bouton, dans l'ordre de la barre.

- **Déplacer un bouton** : glissez la ligne par sa poignée, à gauche.
- **Affiché** : décochez pour masquer un bouton sans le supprimer.

- **Libellé** : le texte du bouton. Deux boutons ne peuvent pas porter le même libellé.
- **Type** et **Cible** : ce que le bouton ouvre. Pour un design, une vue, une page ou un plugin, la cible se choisit dans une liste ; pour un fichier ou un site, elle se saisit.
- **Couleur** : la couleur du texte du bouton, par un nom (red, orange…) ou par la pastille.
- **Au démarrage** : le bouton dont l'onglet est mis au premier plan par **Rétablir les onglets**.
- Le bouton **Ouvrir** (flèche) ouvre la destination dans un nouvel onglet, pour la vérifier.

Rien n'est modifié avant **Enregistrer**. Chaque enregistrement conserve la version précédente. L'extension relit le menu dans la minute qui suit, ou tout de suite par **Relire le menu** dans sa fenêtre. **Reprendre le menu par défaut** remplit le tableau avec le menu fourni par mgDomo, qu'il faut ensuite enregistrer.

Type	Ce que le bouton ouvre
Design	un design de Jeedom
Vue	une vue de Jeedom
Page Jeedom ou plugin	Dashboard, Scénarios, Logs, Historique, Timeline, ou la page d'un plugin
Panneau d'un plugin	le panneau des plugins qui en possèdent un
Fichier du serveur Jeedom	une page HTML du serveur, par son chemin depuis la racine de Jeedom
Site externe, onglet du menu	un site dans un onglet du groupe, avec la barre du menu si le site est autorisé dans l'extension
Site externe, onglet ordinaire	un site dans un onglet à part, sans barre

Fonctionnement

Deux fonctionnements, au choix dans le groupe **Fonctionnement** :

- **Onglets** : un onglet par bouton, rangés dans le groupe « Jeedom », rouverts et mis en sommeil au démarrage du navigateur ; un clic active l'onglet du bouton ou le crée.
- **Classique** (par défaut) : un seul onglet de menu. Le clic charge la page demandée dans l'onglet déjà ouvert par le menu, ou en crée un s'il n'existe pas. Ni groupe, ni rétablissement. Seul l'onglet du menu porte le titre et l'icône du bouton ; un autre onglet Jeedom sur la même page garde la barre et son titre Jeedom. Dans la fenêtre de l'icône, « Rétablir les onglets » laisse place à « Prendre cet onglet comme onglet du menu ».

Le réglage vaut pour tous les navigateurs qui lisent ce menu ; l'extension l'applique à sa relecture suivante.

Barre de boutons : par défaut, la barre n'apparaît que sur les pages du menu (Jeedom et les sites externes des boutons). « Sur tous les sites » l'affiche aussi sur n'importe quel site ouvert dans le navigateur, pour rejoindre le menu d'un clic ; ces pages ne reçoivent qu'une marge de 30 px en haut. L'extension doit alors être autorisée sur tous les sites : bouton « Autoriser » de la fenêtre de l'icône.

Fenêtres du navigateur : par défaut, le menu vit dans **une seule fenêtre**, celle qui contient le groupe d'onglets « Jeedom » (en classique, l'onglet du menu y est rangé) : la barre n'apparaît que dans cette fenêtre, les onglets y sont cherchés et créés, jamais de saut vers une autre fenêtre. Sans groupe nulle part, la première page Jeedom ouverte crée le groupe dans sa fenêtre ; pour changer de fenêtre, glisser le groupe ou « Activer le menu dans cette fenêtre » dans la fenêtre de l'icône. Avec les espaces de travail d'Edge (une fenêtre chacun), c'est ce qui évite que le menu déborde sur les autres espaces. « Toutes les fenêtres » rétablit l'ancien comportement : la barre partout, un onglet du menu commun, et un clic depuis une autre fenêtre fait passer au premier plan celle qui porte l'onglet. Pratique avec un second écran, déroutant avec plusieurs espaces de travail.

Onglet du navigateur

L'onglet de chaque bouton prend une petite icône faite des premières lettres du bouton, pour repérer les onglets du menu d'un coup d'œil. Le groupe **Icône de l'onglet** règle cette icône : nombre de lettres, majuscules, couleurs, et affichage ou non du libellé à côté de l'icône.

L'icône mesure 16 pixels : au-delà de 4 lettres, elles deviennent illisibles.

Sites externes

Un site du type **onglet du menu** reçoit la barre dès qu'il est autorisé dans l'extension (bouton **Autoriser**, section « Sites à autoriser » de la fenêtre de l'icône). Tant qu'il ne l'est pas, son onglet s'ouvre sans barre : rien n'est cassé. Les sites qui refusent d'être affichés dans un cadre ne posent plus de problème, chaque onglet contenant la vraie page.

Problèmes rencontrés

Le bandeau de Jeedom disparaît après un clic sur le logo Jeedom (en haut à gauche) : le logo ouvre la page d'accueil de votre profil Jeedom. Si c'est un design et que l'option « Designs Plein écran » est cochée, Jeedom l'ouvre en plein écran et masque son propre bandeau ; la barre MG Domo reste affichée. L'extension n'y est pour rien : ses boutons n'ouvrent jamais un design en plein écran. Pour garder le bandeau : **Profil** → onglet **Préférences** → **Interface**, décochez **Designs Plein écran**, puis enregistrez.

Réseau privé (Tailscale)

Tailscale relie ce serveur et vos appareils connus (téléphones, PC) en un réseau privé chiffré, sans redirection de port, où qu'ils soient. Hors de la maison, c'est le chemin des pages réservées au réseau local (la carte, la page d'accueil et, avec un routeur de sous-réseau, tout le réseau de la maison). C'est aussi le secours de l'application mgDomo quand l'adresse habituelle de Jeedom ne répond plus : elle l'essaie en dernier, automatiquement.

L'onglet **Réseau privé** se lit en trois étapes. Chaque action lancée ici est journalisée ; rien n'est jamais déconnecté depuis cette page.

1. **Ce serveur** : une phrase dit l'état (Tailscale installé, connecté à votre compte, à jour ?) et propose le geste suivant en un bouton : installer, connecter (un lien à ouvrir dans

votre navigateur) ou mettre à jour. Une action à la fois ; son déroulé s'affiche dans le journal. Les **détails techniques** et les commandes à passer à la main sont repliés.
2. **Vos appareils** : tous les appareils admis dans votre réseau privé. **Tester les liaisons** renseigne la colonne Liaison ; **Clé** dit si la clé de l'appareil est sans échéance, expire à une date ou a expiré ; **Hors ligne depuis** n'est renseigné que pour un appareil hors ligne (au-delà de 7 jours, un téléphone est signalé dans les anomalies). Un appareil inconnu se retire depuis la console Tailscale. La section **Ajouter ou régler un appareil** donne la marche à suivre pour un téléphone Android et pour un PC, puis, pour chaque appareil, le réglage « Disable key expiry » dans la console Tailscale, sans lequel il sort du réseau privé au bout de 180 jours.
3. **Anomalies** : seules les anomalies apparaissent, la liste est vide quand tout est en ordre. Contrôles : démon, DNS, routes, clés, routeur du réseau local, accès aux pages de données, santé Tailscale, téléphone hors ligne depuis plus de 7 jours, téléphone sans utilisateur. Rouge : à corriger, le geste est donné ; orange : à surveiller.

La surveillance continue de ce réseau (démon, clés, routeur, appareils inconnus, téléphone hors ligne, version en retard) et ses alertes sont assurées par le bandeau « Secours Tailscale » de **MG Panoptes** : sans lui, une clé expirée ou un démon arrêté se découvre au moment où l'on en a besoin.

VM/CT Proxmox

L'onglet **VM/CT Proxmox** est une page de conseils : MG Domo n'agit pas sur votre Proxmox. Il décrit comment installer à part, sur votre Proxmox, les services qui alimentent Jeedom par MQTT, pour leur maintenance, leur portabilité et leur souplesse, indépendamment de Jeedom. Les commandes d'installation se collent dans le shell de l'hôte Proxmox ; elles téléchargent et exécutent un script communautaire : lisez-le avant (lien **Source**). Chaque cadre de commande a un bouton **Copier**. Les fichiers d'exemple (corrections de code, configurations) sont téléchargés depuis votre propre Jeedom ; si son adresse interne est en https avec un certificat auto-signé, ajoutez l'option -k aux commandes curl qui les téléchargent.

- **Services MQTT** : un encart d'architecture (appareils, passerelle dans son conteneur, Mosquitto qui sert plusieurs clients, plugin MQTT de Jeedom) avec les plugins MQTT du Market (jMQTT recommandé), puis une fiche par service, avec son usage, la commande d'installation, les réglages qui suivent et les pièges connus :
- **Mosquitto** : le broker, à installer en premier ; un compte par client, adresse fixe.
- **Zigbee2MQTT** et **Z-Wave JS UI** : passerelles Zigbee et Z-Wave, clé USB à donner au conteneur.
- **rfxcom2mqtt** : radio 433 MHz par un RFXtrx (volets Somfy RTS, sondes) ; conteneur Docker, fichiers et 2 corrections de code récupérés depuis Jeedom, topics à publier pour monter, descendre, arrêter et appairer un volet.
- **modbus2mqtt** : registres Modbus d'un onduleur ou d'un compteur ; conteneur Docker, configuration d'exemple pour un onduleur Huawei SUN2000 et 6 corrections de code récupérées depuis Jeedom, topics de lecture, de disponibilité et des 9 registres réglables.
- **Suggestions** : services utiles à côté de Jeedom, même présentation : **Immich** (photothèque locale ; ses albums servent de collections au diaporama de MG Frame TV),

AdGuard Home (filtre DNS de la maison), **Homepage** (page d'accueil du réseau, avec une configuration d'exemple à adapter ; ouverte par le bouton Homepage de l'application MG Domo, elle donne hors de la maison, par Tailscale, un accès « comme à la maison » aux conteneurs et VM du site).

Demander de l'aide

L'onglet **Demande d'intervention** de la page mgDomo propose deux niveaux, à prendre dans l'ordre.

Niveau 1 : joindre un diagnostic

Un fichier décrit l'installation sans rien ouvrir : le rapport technique (versions, santé de Jeedom, plugins mg*, dernières erreurs du plugin choisi), les journaux des plugins mg* et du cœur de Jeedom, la configuration des équipements mg* et les paramètres du socle. Tout est anonymisé avant de sortir : adresses IP et adresses mail masquées, clés, jetons et mots de passe remplacés par `***`, commentaires des équipements exclus.

- Tout se fait à l'étape 3, dans la barre du titre « Message de demande » : choisissez le **plugin concerné** (il donne son nom au fichier, ses dernières erreurs sont mises en tête du rapport et le message le cite ; écosystème entier si aucun n'est choisi), puis **Télécharger le diagnostic** : le fichier `mgDomo_diagnostic_<plugin>_<date>.txt` arrive dans vos téléchargements.
- C'est une archive zip nommée en `.txt`, parce que le forum n'accepte pas les pièces jointes `.zip`. Pour le lire vous-même, renommez-le en `.zip`.
- **Préparer le message** : le type de la demande suit votre dernière action (« Télécharger le diagnostic » = diagnostic, « Créer l'accès » = téléintervention ; sans action depuis l'ouverture de la page, un compte de téléintervention actif = téléintervention) ; il est écrit en première ligne, dans la pastille à côté du message et dans le sujet. Relisez, puis **Envoyer en privé** : le message s'ouvre sur le forum, sujet et texte remplis ; glissez-y le fichier, puis envoyez.

Niveau 2 : la téléintervention

Quand le problème ne se lit pas sur pièces (comportement intermittent, matériel, réseau), vous ouvrez un accès temporaire : vérification de l'accès distant, création d'un compte administrateur (mot de passe affiché une seule fois), message privé sur le forum, puis révocation dès la fin de l'intervention. Le compte est de toute façon supprimé automatiquement au bout de 7 jours.

À propos

- **Auteur** : Michel Georgein. L'écosystème MG Domo est né d'une installation domotique réelle, utilisée au quotidien, puis ouvert à d'autres utilisateurs de Jeedom.
- **Licence** : GNU Affero GPL (AGPL), pour MG Domo et tous les plugins mg*.
- **Distribution** : les plugins sont publiés sur GitHub, dépôt `georgein/mgdomo-dist`. MG Domo s'installe par la commande donnée dans sa page de configuration (encart «

Rappel : installer MG Domo sur une autre box Jeedom ») ; les autres plugins s'installent et se mettent à jour depuis l'onglet **Plugins**, avec contrôle d'intégrité.

- **Documentation** : l'aide de chaque plugin s'ouvre par le « ? » de sa page ; ses nouveautés par le bouton **Changelog** de l'onglet Plugins. La même aide est publiée en PDF, qui s'ouvrent directement dans le navigateur, avec un PDF d'ensemble (présentation, installation, liste des plugins) : https://georgein.github.io/mgdomo-dist/
- **Composants tiers** : la liste des bibliothèques embarquées, avec leur source et leur version, est dans l'onglet **Bibliothèques**.
- **Aide et contact** : sur le forum Jeedom, par le bouton **Community** (sujet public pré-rempli) ou par l'onglet **Demande d'intervention** (message privé à l'auteur, avec diagnostic anonymisé ou accès temporaire).
- **Soutien** : un don libre est possible sur ko-fi.com/mg_domo (barre en bas de l'onglet Demande d'intervention).

Classe mg : aide sur les fonctions

La **classe mg** est la bibliothèque de fonctions du socle MG Domo. Elle simplifie l'écriture des scénarios (lire ou écrire une commande, envoyer un message, gérer une alerte, calculer une durée...) et sert de base commune à tous les plugins mg*. Cette page décrit toutes les fonctions que vous pouvez appeler ; les rouages internes de la classe n'y figurent pas, ils ne sont pas appelables de l'extérieur.

Utilisation

- **Dans un bloc de code de scénario** : appelez la fonction précédée de `mg::`, par exemple `mg::message('', 'Bonjour');`.
- **Option « Accessible partout »** (configuration du plugin MG Domo, groupe « Classe mg ») : cochée, la classe est chargée toute seule, partout (scénarios, plugins, code personnel), sans rien inclure.
- **Option décochée** : ajoutez en tête du bloc de code `include_once getRootPath() . '/plugins/mgDomo/core/class/mg.class.php'; mg::init();`.
- **Dans le calcul d'une commande virtuelle** (option cochée) : le nom de la fonction **sans** `mg::` et les arguments **sans guillemets**, par exemple `getConfigJeedom(core,internalAddr)` ; l'équipement **Démo widgets** (objet Exemples MG Domo) en montre trois.
- **Commandes** : une commande se désigne par son nom lisible `#[Objet][Equipement][Commande]#` ou par son identifiant `#1234#` ; les formes exactes acceptées sont détaillées à `mg::getCmd`.
- **Journal** : `mg::message` et les fonctions qui s'en servent écrivent dans le log du scénario qui les appelle ; le niveau de détail se règle avec `mg::setNiveauDebug()`.
- Les fonctions marquées **réservée aux plugins mgDomo** servent aux plugins de l'écosystème ; elles restent appelables, mais n'ont pas d'usage courant dans un scénario.

Sommaire

Cliquez sur un thème ou sur une fonction pour y aller ; chaque thème se termine par un lien de retour au sommaire.

- **1. Messages et journaux** : message, messageT, debug, test, setmessageJeedom, alertJeedom, popupJeedom, addLog, addLogStatic
- **2. Alertes** : alertes, getAlertes, rappelAlertes, destUtilisateur
- **3. Lire et piloter les commandes** : getCmd, setCmd, setInf, existCmd, mkCmd, getInfoCmd, getCmdUnite, getMinMaxCmd, setMinMaxCmd, getParamWidget, setParamWidget, toHuman, toID, extractPartCmd, getExp, isVisible
- **4. Historique et mouvements** : StatsHisto, durationBetween, lastMvmt, minuterie
- **5. Équipements** : getEquipement, setEquipement, getConfigEquipement, ConfigEquiLogic, isActive, getTypeEqui, clone_Equip
- **6. Variables et tags** : getVar, setVar, unsetVar, getTag, setTag

- **7. Scénarios** : getScenario, setScenario, stopScenario, scenario_return, setCron, cron2Time, Declencheur, wait, setCmdWait, EvaluateCondition
- **8. Réglages, tables et utilisateurs** : getParam, getValSql, getTabSql, ensureSeedData, getConfigJeedom, userPhoto, userCouleur, getDomoApiKey
- **9. Dates, durées et calculs** : formateDuree, TimeBetween, Soleil, getDistance, getAltitude, getAltitudeBatch, tri_Tableau, getICS
- **10. Voix, médias, mails et réseau** : VoiceRSS, etatMedia, ensureMedia, securiserPartageMedia, mail_PJ, WakeOnLan, internetCoupe, openAI, anthropic, convoyeurApk
- **11. Système et dépannage** : NettoieRep, jeedom_poweroff, masquerCles, rapportDepannage, paquetDiagnostic, pluginName

---

1. Messages et journaux

mg::message

`mg::message($type, $contenu = '', $titre = 'info')` Envoie un message sur un ou plusieurs canaux en un seul appel : log du scénario, log nommé, centre de messages Jeedom, SMS, mail, annonce vocale, son ou notification sur téléphone.

- **Paramètres** : `$type` : liste de groupes séparés par des virgules (détail ci-dessous), ou numéro de scénario (entier) ; `$contenu` : le texte (vide = WARNING, rien n'est émis) ; `$titre` : selon le cas un code de style des logs, un volume (TTS, PLAY) ou le sujet (MAIL, NOTIF) (défaut `'info'`).
- **Retour** : aucun.
- **Exemple** : `mg::message('LOG:_Timeline, SMS:@AB, MAIL:@AB', 'Porte du garage ouverte', 'Alerte garage');`
- **Remarque** : un groupe s'écrit `CANAL`, `CANAL:DEST` ou `CANAL:CONVOYEUR:DEST` ; plusieurs destinataires d'un même groupe se séparent par `/` (ex. `SMS:@AB/@CD`). Canal et convoyeur s'écrivent indifféremment en majuscules ou minuscules.
- **Remarque** : canaux : `LOG` log du scénario courant (`LOG` ou `LOG:x`) ou log Jeedom nommé (`LOG:_Timeline`) ; `MESSAGE` centre de messages Jeedom (sans destinataire) ; `SMS:dest` ; `MAIL:dest` ; `TTS:dest` annonce vocale ; `PLAY:dest` son (nom de fichier ou favori) ; `NOTIF:dest` notification sur le téléphone.
- **Remarque** : destinataires : `@code` = personne de l'annuaire mgDomo, traduite selon le canal (SMS : son numéro ; MAIL : son adresse ; TTS, PLAY, NOTIF : son téléphone) ; une valeur directe (numéro, adresse mail) pour SMS et MAIL ; `DEFAUT` = la maison (enceintes) pour TTS, PLAY, et le téléphone passerelle pour NOTIF. Pour TTS, PLAY et NOTIF, seuls `DEFAUT` et `@code` sont acceptés.
- **Remarque** : convoyeurs (qui émet) : `TTSCast` (enceinte Google), `SONOS` (enceinte Sonos), `apk` (téléphone mgDomo : voix, SMS, mail, notification), `Mail` (plugin Mail de Jeedom). Sans convoyeur nommé, le défaut du canal est lu dans les réglages Media : SMS → apk, MAIL → Mail, TTS et PLAY → TTSCast, NOTIF → apk. Les enceintes ne visent que la maison ; pour parler sur un téléphone, nommer apk : `TTS:apk:@AB`. Le son (PLAY) n'est servi que par les enceintes.
- **Remarque** : codes de format (3e paramètre) : `'bg xxx'` ou `'fg xxx'` (xxx = success, info, warning, danger) ou `'b'`, `'i'`, `'s'` habillent les lignes de log seulement ; les

canaux humains reçoivent le texte brut. Pour MAIL, si `$titre` est un code de style le sujet devient la 1re ligne du texte (78 caractères) ; pour TTS et PLAY un nombre dans `$titre` est le volume (0 = volume par défaut) ; un `@` en tête du texte joue un jingle avant l'annonce sur les enceintes.
- **Remarque** : si aucun groupe `LOG` n'est nommé, le message est aussi écrit dans le log du scénario courant. Le texte du log est filtré par le niveau fixé par `mg::setNiveauDebug` ; un texte sans niveau est traité comme `INFO :` ; préfixer `ERROR :`, `WARNING :` ou `INF2 :` pour le classer.
- **Remarque** : `$type` entier positif (ex. `mg::message(42, 'Texte')`) écrit directement dans le log du scénario 42, sans filtre ni autre canal ; un numéro écrit en texte (`'42'`) est accepté de la même façon. Canal, convoyeur ou destinataire inconnu : ERROR au log, ce groupe n'est pas émis, les autres le sont. Les enceintes refusent l'émission depuis une page web et pendant une coupure internet.

mg::messageT

`mg::messageT($type, $contenu = '', $format = 'fg info')` Écrit un message encadré façon titre (`===== TITRE =====`), sur 138 caractères, avec une ligne pleine au-dessus et au-dessous ; sert à baliser les étapes dans un log.

- **Paramètres** : `$type` : destination, même syntaxe que `mg::message` (défaut habituel `''` = log du scénario) ; `$contenu` : le texte, dont le premier caractère peut être un drapeau ; `$format` : style de log `'bg xxx'`, `'fg xxx'`, `'b'`, `'i'`, `'s'` avec xxx = success, info, warning ou danger (défaut `'fg info'`).
- **Retour** : aucun.
- **Exemple** : `mg::messageT('', "! DEBUT DE L'ARROSAGE", 'bg success');`
- **Remarque** : drapeaux en tête du texte : `!` lignes haute et basse sans couleur ; `.` pas de lignes haute et basse ; `?` suivi d'un caractère : ce caractère remplace `=` pour tracer les lignes ; espace : une seule ligne, sans remplissage ; tout autre début : lignes en couleur.
- **Remarque** : chaque ligne est envoyée par `mg::message`, donc vers les mêmes canaux que `$type` (à réserver aux logs).

mg::setNiveauDebug

`mg::setNiveauDebug($debug = 5, $mode = 'Manuel')` Règle le niveau de détail des messages écrits par `mg::message` dans les logs.

- **Paramètres** : `$debug` : niveau (défaut 5 = tout) : 0 rien, 1 INFO, 2 INFO et ERROR, 3 + WARNING, 4 et plus + INF2 ; `$mode` : conservé pour compatibilité, sans effet (défaut `'Manuel'`).
- **Retour** : aucun.
- **Exemple** : `mg::setNiveauDebug(3);`
- **Remarque** : n'écrit rien lui-même ; les messages INF2 demandent en plus que le log détaillé soit activé pour le scénario.
- **Remarque** : au démarrage d'un scénario, le niveau est celui du réglage `log / debugLevel` de la configuration mgDomo (4 par défaut) ; `mg::setNiveauDebug()` le change pour la suite du scénario seulement.

mg::tester

`mg::tester()` Affiche dans le log du scénario un échantillon de toutes les mises en forme possibles des logs (titres encadrés, pastilles de niveau, couleurs de fond et de texte, gras, italique, texte caché).

- **Paramètres** : aucun.
- **Retour** : aucun.
- **Exemple** : `mg::tester();`

mg::setMessageJeedom

`mg::setMessageJeedom($message)` Ajoute un message dans le centre de messages de Jeedom.

- **Paramètres** : `$message` : texte du message (`'0'` est accepté).
- **Retour** : `true` si le message est ajouté ; rien si le message est vide.
- **Exemple** : `mg::setMessageJeedom('Pile faible sur le détecteur du garage');`

mg::alerterJeedom

`mg::alerterJeedom($message, $level = 'info')` Affiche un bandeau d'alerte dans tous les navigateurs où une page Jeedom est ouverte.

- **Paramètres** : `$message` : texte de l'alerte ; `$level` : `info`, `success`, `warning` ou `danger` (défaut `info`).
- **Retour** : aucun.
- **Exemple** : `mg::alerterJeedom('Fenêtre ouverte avec chauffage en marche', 'warning');`
- **Remarque** : rien n'est affiché si aucune page Jeedom n'est ouverte ; un niveau inconnu est refusé (message d'erreur dans le log).

mg::afficherPopupJeedom

`mg::afficherPopupJeedom($message)` Affiche une fenêtre qui doit être validée, dans tous les navigateurs où une page Jeedom est ouverte.

- **Paramètres** : `$message` : texte de la fenêtre.
- **Retour** : aucun.
- **Exemple** : `mg::afficherPopupJeedom('Pensez à fermer le portail');`
- **Remarque** : rien n'est affiché si aucune page Jeedom n'est ouverte.

mg::addLog

`mg::addLog($level = 'info', $style = null, $message = '*** AUCUN MESSAGE ***', $eqLogic = null)` Écrit une ligne dans le log du plugin appelant, préfixée du nom de l'équipement concerné et de la fonction appelante.

- **Paramètres** : `$level` : `'debug'`, `'info'`, `'warning'` ou `'error'` (défaut `'info'`, toute autre valeur devient `'info'`) ; `$style` : style Jeedom optionnel, ex. `'bg-info'`, `'fg-warning'` (défaut `null`) ; `$message` : le texte (défaut `'*** AUCUN MESSAGE ***'`) ; `$eqLogic` :

équipement à utiliser pour le log et le préfixe, prioritaire sur la détection automatique (défaut `null`).
- **Retour** : aucun.
- **Exemple** : `mg::addLog('warning', 'fg-warning', 'Capteur muet depuis 2 h', $eqLogic);`
- **Usage** : réservée aux plugins mgDomo (appel depuis une classe de plugin ; depuis un scénario la ligne irait dans un log nommé `mg`).
- **Remarque** : si l'équipement a l'option « log actif » à 0, les niveaux debug et info sont ignorés ; warning et error sont toujours écrits. Sans équipement, le préfixe est `[Static]`.

mg::addLogStatic

`mg::addLogStatic($level = 'info', $style = null, $message = '*** AUCUN MESSAGE ***')`
Variante de `mg::addLog` pour les traitements sans équipement (tâches planifiées, écouteurs, démons) : la ligne est préfixée `[GLOBAL]`.

- **Paramètres** : `$level` : `'debug'`, `'info'`, `'warning'` ou `'error'` (défaut `'info'`) ; `$style` : style Jeedom optionnel (défaut `null`) ; `$message` : le texte (défaut `'*** AUCUN MESSAGE ***'`).
- **Retour** : aucun.
- **Exemple** : `mg::addLogStatic('info', null, 'Synchronisation nocturne terminée');`
- **Usage** : réservée aux plugins mgDomo (le log visé est celui de la classe appelante).
- **Remarque** : pas de filtre « log actif » (aucun équipement en jeu).

Retour au sommaire

---

2. Alertes

mg::alertes

`mg::alertes($name = '', $message = '', $destinataires = '', $periodicite = 0, $timeout=-1, $color = '')` Crée, modifie ou supprime une alerte répétée. Une alerte créée est envoyée une première fois, puis rappelée à intervalle régulier jusqu'à sa suppression ou l'expiration de sa durée, par `mg::rappelerAlertes()` ; elle peut allumer une lampe-voyant de couleur.

- **Paramètres** : `$name` : nom de l'alerte (obligatoire ; accents retirés, 50 caractères au plus) ; `$message` : texte de l'alerte (défaut à la création : « Message ») ; `$destinataires` : destinataires au format de `mg::message()` (défaut à la création : `log:`) ; `$periodicite` : intervalle des rappels en minutes (défaut 0) ; `$timeout` : durée de vie de l'alerte en minutes (défaut -1 = non fourni, soit 30 minutes à la création) ; `$color` : couleur du voyant parmi `ROUGE`, `ROSE`, `ORANGE`, `MAGENTA`, `BLEU`, `JAUNE`, `CYAN`, `VERT`, `BLANC`, ou `NOIR` pour une alerte sans voyant (défaut `BLANC` à la création).
- **Retour** : aucun.
- **Exemple** : `mg::alertes('Congelateur', 'Le congélateur est trop chaud', 'sms:moi, tts:', 15, 120, 'BLEU');`
- **Exemple** : `mg::alertes('Congelateur');` (supprime l'alerte)
- **Remarque** : sur une alerte existante, seuls les paramètres fournis changent ('' , 0 ou -1 = garder la valeur actuelle) ; une modification n'entraîne pas de nouvel envoi immédiat,

et un appel répété avec les mêmes valeurs ne change rien. Un changement de texte remet le compteur de rappels à zéro.
- **Remarque** : l'appel avec le seul nom supprime l'alerte et éteint la lampe-voyant s'il ne reste aucune autre alerte avec voyant. Un timeout de 0 compte comme non fourni (30 minutes à la création, durée conservée sur une alerte existante) ; une couleur inconnue est ignorée.

mg::getAlertes

`mg::getAlertes($name = '')` Lit les alertes en cours, toutes ou une seule.

- **Paramètres** : `$name` : nom d'une alerte précise (défaut '' = toutes les alertes).
- **Retour** : avec un nom : tableau de l'alerte (`message`, `destinataires`, `periodicite`, `timeOut`, `color`, `debut`, `rappelFait`, `lastNotif`, `compteur`, `isNew`) ou `null` si elle n'existe pas ; sans nom : tableau indexé par nom d'alerte, trié par date de début (vide s'il n'y en a aucune).
- **Exemple** : `if (mg::getAlertes('Congelateur') !== null) mg::message('', 'Alerte congélateur toujours en cours');`
- **Remarque** : en cas de lecture impossible de la table des alertes, une erreur est journalisée et la fonction répond comme s'il n'y avait aucune alerte.

mg::rappelerAlertes

`mg::rappelerAlertes($nameAppel = '', $silencieux = false)` Traite les alertes en cours : envoie la première émission des alertes nouvelles, les rappels arrivés à échéance, supprime les alertes expirées et pilote la lampe-voyant. Le plugin MG Domo l'appelle lui-même : aussitôt qu'une alerte est créée, modifiée ou supprimée, puis chaque minute (tâche dédiée, mode silencieux quand la variable `nePasDeranger` vaut 1). Aucun scénario n'est nécessaire. Un appel par un scénario reste possible : une seule passe tourne à la fois, un appel pendant une passe en cours ne fait rien.

- **Paramètres** : `$nameAppel` : nom d'une alerte à SUPPRIMER pendant ce passage ('' = aucune ; les autres alertes sont traitées normalement) (défaut ''); `$silencieux` : mode nuit, `true` = seulement les envois écrits (SMS, mail, notifications, journaux), sans voix ni son ni lampe-voyant (défaut `false`).
- **Retour** : aucun.
- **Exemple** : `mg::rappelerAlertes('', mg::getVar('nePasDeranger', 0) == 1);`
- **Remarque** : pour les annonces vocales, le texte est reformulé par l'IA Claude sur un ton aléatoire pour varier les rappels (texte d'origine conservé si l'IA échoue, si internet est coupé ou si la reformulation dépasse 250 caractères) ; SMS, mails et journaux reçoivent toujours le texte d'origine. Un rappel manqué parce qu'un passage précédent était occupé est rattrapé au passage suivant.
- **Remarque** : les paramètres `equip` / `equipLampeAlerte` (lampe-voyant), `equip` / `equipSalonResume` (sa commande Luminosité règle l'intensité de la lampe) et `equip` / `equipMaisonGeneral` (commandes Alarme_Périodique et Alarme_Message, dernière alerte émise) sont facultatifs : valeur vide = fonction correspondante absente, les alertes partent quand même. La couleur affichée est celle de l'alerte la plus grave en cours.

mg::getDestinationsUtilisateurs

`mg::getDestinationsUtilisateurs($canaux = 'SMS, Mail')` Rend les destinations de `mg::message()` vers l'utilisateur principal de la maison : celui dont la case « Utilisateur principal » est cochée dans sa fiche (onglet Utilisateurs de MG Domo) ; aucun coché : le premier utilisateur qui habite la maison, ceux qui ont un mail d'abord, puis par nom.

- **Paramètres** : `$canaux` : canaux voulus, séparés par des virgules (défaut `'SMS, Mail'`).
- **Retour** : par exemple `SMS:@U1, Mail:@U1` ; chaîne vide si aucun utilisateur actif n'est déclaré.
- **Exemple** : `mg::message(trim('Log:/_Timeline, ' . mg::getDestinationsUtilisateurs(), ', '), 'Porte de garage restée ouverte');`

Retour au sommaire

---

3. Lire et piloter les commandes

mg::getCmd

`mg::getCmd($cmd, $complement = '', &$collectDate = '', &$valueDate = '', $xxx=true, $defaut = null, $forceDefaut = false)` Lit la valeur d'une commande info, convertie selon son sous-type, et donne si besoin ses dates de collecte et de changement.

- **Paramètres** : `$cmd` : la commande (formes acceptées ci-dessous), ou l'équipement si `$complement` est renseigné ; `$complement` : nom de la commande dans l'équipement (défaut vide) ; `&$collectDate` : reçoit la date de dernière collecte (timestamp) ; `&$valueDate` : reçoit la date du dernier changement de valeur (timestamp) ; `$xxx` : `true` trace la lecture dans le log, `false` lit en silence (défaut `true`) ; `$defaut` : valeur rendue en cas d'échec (défaut `null`) ; `$forceDefaut` : `true` pour qu'une valeur vide soit remplacée par `$defaut` ET que `$defaut` soit écrit dans la commande (défaut `false`).
- **Formes acceptées pour la commande** : `'#[Objet][Equipement][Commande]#'` ; `'[Objet][Equipement][Commande]'` (les `#` sont ajoutés) ; id avec ou sans `#` (`'#1234#'`, `'1234'`, `1234`) ; ou couple équipement + `$complement`, l'équipement étant `'#[Objet][Equipement]#'`, `'[Objet][Equipement]'` ou son id (`12`, `'#12#'`, `'eqLogic12'`, `'#eqLogic12#'`) et le complément le nom de la commande avec ou sans `[]`.
- **Retour** : sous-type numérique : un entier ou un décimal (une valeur non numérique, vide comprise, rend `$defaut`) ; binaire : `0` ou `1` (une valeur vide rend `0`, ou `$defaut` si `$forceDefaut`) ; texte : une chaîne (vide si vide) ; `$defaut` si la commande est vide, introuvable ou n'est pas de type info (les deux dates valent alors `0`).
- **Exemple** : `$temp = mg::getCmd('#[Salon][Thermomètre][Température]#', '', $collecte, $maj); if (time() - $maj > 3600) { ... }`
- **Exemple** : `$mode = mg::getCmd('#[Maison][Virtuel Mode][Mode]#', '', $_, $_, false, 'Jour', true);`
- **Remarque** : `$forceDefaut = true` modifie la commande en base (écriture de `$defaut`), ce qui peut déclencher les scénarios qui la surveillent ; une référence non résolue ou un id qui n'est pas fait uniquement de chiffres est traité comme introuvable.

mg::setCmd

`mg::setCmd($cmd, $complement = '', $value = '', $title = '', $recipient = '')` Exécute une commande d'action (bouton, curseur, couleur, liste, message) en adaptant la valeur envoyée au sous-type de la commande.

- **Paramètres** : `$cmd` : la commande (mêmes formes que pour `mg::getCmd`), ou l'équipement si `$complement` est renseigné ; `$complement` : nom de la commande (défaut vide) ; `$value` : valeur à envoyer : nombre pour un curseur, couleur `#RRGGBB` pour une couleur, valeur ou libellé de la liste pour un choix, texte pour un message (défaut vide) ; `$title` : titre d'un message, ou options facultatives transmises à une commande de type liste (défaut vide) ; `$recipient` : destinataire, pour les commandes de message qui l'acceptent comme l'envoi de mail à une adresse libre (défaut vide).
- **Formes acceptées pour la commande** : `'#[Objet][Equipement][Commande]#'`, `'[Objet][Equipement][Commande]'`, id avec ou sans `#`, ou équipement (`'#[Objet][Equipement]#'`, `'[Objet][Equipement]'`, `12`, `'#12#'`, `'eqLogic12'`, `'#eqLogic12#'`) + nom de commande dans `$complement`.
- **Retour** : `true` si la commande a été lancée, `false` si elle est introuvable, de type info, de sous-type inconnu, si la valeur d'un curseur n'est pas un nombre, si la valeur d'une liste n'existe pas, ou en cas d'erreur.
- **Exemple** : `mg::setCmd('#[Salon][Lampe]#', 'On');`
- **Exemple** : `mg::setCmd('#[Salon][Volet]#', 'Positionnement', 30);`
- **Exemple** : `mg::setCmd('#[Maison][Notification][Envoyer]#', '', 'Il fait #[Salon][Thermomètre][Température]# °C', 'Température');`
- **Remarque** : la valeur d'un curseur, d'une couleur ou d'une liste est évaluée comme une expression Jeedom ; le titre et le texte d'un message voient seulement leurs références de commandes et leurs tags remplacés par leur valeur, sans aucune fonction exécutée ; une liste se choisit par sa valeur ou par son libellé, sans tenir compte de la casse ; après `mg::setAttenteCmd(false)`, la commande part sans attendre de réponse.

mg::setInf

`mg::setInf($cmd, $complement = '', $value = '')` Force directement la valeur d'une commande info (par exemple d'un virtuel), sans passer par une commande d'action.

- **Paramètres** : `$cmd` : commande info, ou équipement si `$complement` est renseigné ; `$complement` : nom de la commande (défaut vide) ; `$value` : valeur à écrire (défaut chaîne vide).
- **Retour** : aucun.
- **Exemple** : `mg::setInf('#[Maison][Virtuel Mode][Mode]#', '', 'Nuit');`
- **Remarque** : refuse (avec message d'erreur dans le log) une commande introuvable ou qui n'est pas de type info ; la nouvelle valeur déclenche les scénarios qui surveillent la commande.

mg::existCmd

`mg::existCmd($cmd, $complement = '', &$type = '')` Vérifie qu'une commande existe et donne son id et son type.

- **Paramètres** : `$cmd` : commande, ou équipement si `$complement` est renseigné ; `$complement` : nom de la commande dans l'équipement (défaut vide) ; `&$type` : variable qui reçoit le type (`info` ou `action`), ou `Introuvable` / `Sans type` en cas d'échec.
- **Retour** : l'id de la commande si elle existe, `false` sinon.
- **Exemple** : `if (mg::existCmd('#[Salon][Lampe]#', 'Etat', $type)) { ... }`

mg::mkCmd

`mg::mkCmd($equipement, $complement)` Construit la référence complète d'une commande à partir d'un équipement et du nom de la commande.

- **Paramètres** : `$equipement` : équipement sous la forme `[Objet][Equipement]` (avec ou sans `#`) ou par son id (`12`, `#12#`, `eqLogic12`, `#eqLogic12#`) ; `$complement` : nom de la commande, avec ou sans `[]` ou `#`.
- **Retour** : la référence `#[Objet][Equipement][Commande]#` ; si `$complement` est vide, `$equipement` est renvoyé tel quel.
- **Exemple** : `$cmd = mg::mkCmd('#[Salon][Lampe]#', 'Etat');`
- **Remarque** : si l'équipement désigné par un id n'existe pas, le résultat garde la forme `#eqLogicN[Commande]#` et ne désigne aucune commande (les fonctions de lecture ou d'action le traitent comme introuvable).

mg::getInfoCmd

`mg::getInfoCmd($cmd)` Retrouve la commande d'état (info) liée à une commande d'action, par exemple l'état associé au bouton On d'une lampe.

- **Paramètres** : `$cmd` : commande d'action, par sa référence `#[Objet][Equipement][Commande]#` ou son id (avec ou sans `#`).
- **Retour** : la référence `#[Objet][Equipement][Commande]#` de la commande info liée ; `null` si la commande est introuvable, déjà de type info ou sans commande liée ; `false` si `$cmd` est vide.
- **Exemple** : `$etat = mg::getInfoCmd('#[Salon][Lampe][On]#');`

mg::getCmdUnite

`mg::getCmdUnite($cmd, $complement = '')` Donne l'unité d'une commande (°C, W, %...).

- **Paramètres** : `$cmd` : commande, ou équipement si `$complement` est renseigné ; `$complement` : nom de la commande (défaut vide).
- **Retour** : l'unité en texte ; chaîne vide si aucune unité ou commande introuvable.
- **Exemple** : `$unite = mg::getCmdUnite('#[Salon][Thermomètre][Température]#');`

mg::getMinMaxCmd

`mg::getMinMaxCmd($cmd, $complement = '', $min_max = 'max')` Donne la valeur minimale ou maximale configurée sur une commande.

- **Paramètres** : `$cmd` : commande, ou équipement si `$complement` est renseigné ; `$complement` : nom de la commande dans l'équipement (défaut vide) ; `$min_max` : `'min'` ou `'max'` (défaut `'max'`, toute autre valeur vaut `'max'`).

- **Retour** : la valeur configurée ; `null` si la commande est introuvable.
- **Exemple** : `$max = mg::getMinMaxCmd('#[Salon][Thermostat]#', 'Consigne', 'max');`
- **Remarque** : une commande introuvable ajoute aussi un message dans le centre de messages Jeedom.

mg::setMinMaxCmd

`mg::setMinMaxCmd($cmd, $complement, $min, $max)` Change les valeurs minimale et maximale configurées sur une commande.

- **Paramètres** : `$cmd` : commande, ou équipement si `$complement` est renseigné ; `$complement` : nom de la commande, ou `''` ; `$min` : nouvelle valeur minimale ; `$max` : nouvelle valeur maximale.
- **Retour** : `true` si la modification est faite, `false` si la commande est introuvable.
- **Exemple** : `mg::setMinMaxCmd('#[Salon][Thermostat][Consigne]#', '', 15, 25);`
- **Remarque** : une commande introuvable ajoute aussi un message dans le centre de messages Jeedom.

mg::getParamWidget

`mg::getParamWidget($cmd, $complement = '', $clef = null, $defaut = null)` Lit un paramètre optionnel du widget d'une commande (réglage saisi dans la configuration d'affichage de la commande).

- **Paramètres** : `$cmd` : commande, ou équipement si `$complement` est renseigné ; `$complement` : nom de la commande (défaut vide) ; `$clef` : nom du paramètre à lire ; `$defaut` : valeur rendue si le paramètre est absent ou vide (défaut `null`).
- **Retour** : la valeur du paramètre (une valeur `0` est bien rendue) ; `$defaut` si le paramètre est absent ou vide, si `$clef` est `null`, si la commande est introuvable ou si `$cmd` est vide.
- **Exemple** : `$seuil = mg::getParamWidget('#[Piscine][Pompe][Durée]#', '', 'seuilAlerte', 120);`

mg::setParamWidget

`mg::setParamWidget($cmd, $complement = '', $clef = null, $value = null, $refresh = true)` Crée ou modifie un paramètre optionnel du widget d'une commande, et rafraîchit la tuile pour que le changement soit visible sans recharger la page.

- **Paramètres** : `$cmd` : commande, ou équipement si `$complement` est renseigné ; `$complement` : nom de la commande (défaut vide) ; `$clef` : nom du paramètre ; `$value` : nouvelle valeur ; `$refresh` : `true` pour redessiner la tuile après un changement, `false` pour écrire sans redessiner (défaut `true`).
- **Retour** : `true` si la valeur a changé et a été enregistrée, `false` si elle était déjà identique, `null` si la commande est vide ou introuvable.
- **Exemple** : `mg::setParamWidget('#[Maison][Jauge][Niveau]#', '', 'couleur', 'vert');`
- **Remarque** : utile pour faire varier un réglage de widget selon une valeur calculée ; `$refresh = false` convient à un paramètre qui change très souvent.

mg::toHuman

`mg::toHuman($texte, $complement = '')` Remplace dans un texte les références par id (`#1234#`, `#eqLogic12#`) par les noms lisibles des commandes et équipements.

- **Paramètres** : `$texte` : texte contenant des références ; `$complement` : si renseigné, `$texte` est un équipement et `$complement` le nom de la commande (défaut vide).
- **Retour** : le texte avec les noms `#[Objet][Equipement][Commande]#` à la place des id ; le texte inchangé s'il ne contient aucun `#`.
- **Exemple** : `$nom = mg::toHuman('#1234#');`
- **Remarque** : un `#` sans lien avec une référence (couleur `#FF0000`, `Ticket #42`) est laissé tel quel.

mg::convertirEnId

`mg::convertirEnId($texte, $complement = '', $msg = null)` Inverse de `mg::toHuman` : convertit la référence lisible d'une commande ou d'un équipement en référence par id.

- **Paramètres** : `$texte` : une référence `#[Objet][Equipement][Commande]#` ou `#[Objet][Equipement]#` ; `$complement` : si renseigné, `$texte` est un équipement et `$complement` le nom de la commande (défaut vide) ; `$msg` : valeur vraie pour tracer le résultat dans le log (défaut `null`).
- **Retour** : la référence par id avec ses `#` (ex. `#1234#`, ou `#eqLogic12#` pour un équipement) ; chaîne vide si `$texte` est vide.
- **Exemple** : `$id = intval(trim(mg::convertirEnId('#[Salon][Lampe][Etat]#'), '#'));`
- **Remarque** : ne traiter qu'une seule référence à la fois ; sur un texte qui en contient plusieurs, les références du début et de la fin sont mal converties : utiliser plutôt `mg::getExp` pour une expression.

mg::extrairePartieCmd

`mg::extrairePartieCmd($cmd, $position = 3)` Extrait un segment (objet, équipement ou commande) de la référence d'une commande ou d'un équipement.

- **Paramètres** : `$cmd` : référence (ou id avec `#`) d'une commande ou d'un équipement ; `$position` : 1 objet, 2 équipement, 3 commande (défaut 3).
- **Retour** : le segment demandé, en texte ; si `$cmd` ne contient pas de `[...]`, le texte sans ses `#` ; `null` si la position n'existe pas (0 ou au-delà du nombre de segments).
- **Exemple** : `$piece = mg::extrairePartieCmd('#[Salon][Lampe][Etat]#', 1);`

mg::getExp

`mg::getExp($exp)` Évalue une expression avec le moteur d'expression de Jeedom, comme le testeur d'expression.

- **Paramètres** : `$exp` : l'expression ; elle peut contenir des commandes `#[Objet][Equipement][Commande]#`, des tags, les fonctions Jeedom (`variable()`, `scenario()`, `average()`...) et des calculs.
- **Retour** : le résultat de l'évaluation ; `null` si l'expression est vide.
- **Exemple** : `$moy = mg::getExp('average(#[Salon][Thermomètre][Température]#, 1 hour)');`

- **Remarque** : les fonctions présentes dans le texte sont réellement exécutées : ne jamais y passer un texte d'origine extérieure (agenda, message reçu).

mg::estVisible

`mg::estVisible($cmd_or_eqLogic)` Indique si une commande ou un équipement est visible.

- **Paramètres** : `$cmd_or_eqLogic` : commande ou équipement, par sa référence ou son id ; la recherche se fait d'abord parmi les commandes, puis parmi les équipements.
- **Retour** : `true` si visible, `false` si masqué, introuvable ou paramètre vide.
- **Exemple** : `if (!mg::estVisible('#[Salon][Lampe]#')) mg::setEquipement('#[Salon][Lampe]#', 'show');`
- **Remarque** : un id numérique seul est d'abord cherché comme commande ; pour un équipement par id, préférer la forme `#eqLogic12#`.

Retour au sommaire

---

4. Historique et mouvements

mg::getStatsHistorique

`mg::getStatsHistorique($idCmd, $result = 'D', $typePeriode = 'M', $deltaStart = 0, $nbPeriodes = 1)` Extrait de l'historique d'une commande une valeur sur une période calendaire : valeur de début, de fin, écart entre les deux (idéal pour un compteur d'index) ou durée passée à 1.

- **Paramètres** : `$idCmd` : identifiant numérique de la commande historisée (`123` ou `'#123#'`) ; `$result` : `'S'` valeur de départ, `'E'` valeur de fin, `'D'` différence fin moins début, `'DB'` durée à l'état 1 (défaut `'D'`) ; `$typePeriode` : `'T'` heure, `'D'` jour, `'M'` mois, `'Y'` année (défaut `'M'`) ; `$deltaStart` : nombre de périodes à reculer depuis la période en cours pour fixer le début (défaut 0 = période en cours) ; `$nbPeriodes` : nombre de périodes couvertes à partir de ce début, vers l'avenir (défaut 1, doit être > 0).
- **Retour** : un nombre (valeurs arrondies au dixième par Jeedom ; `'DB'` en secondes) ; 0 si une borne n'a aucune donnée ou si un paramètre est invalide.
- **Exemple** : `$consoMoisDernier = mg::getStatsHistorique(1234, 'D', 'M', 1, 1);`
- **Remarque** : la période part du début de la période visée et s'étend vers l'avant : pour les 12 derniers mois complets, écrire `mg::getStatsHistorique(1234, 'D', 'M', 12, 12)` ; `(..., 'M', 0, 12)` couvre le mois en cours et les 11 suivants. La commande doit être historisée.
- **Remarque** : en mode `'D'`, la valeur de départ est le dernier relevé avant le début de période ; à défaut, le relevé le plus ancien antérieur, sinon le premier relevé de la période.

mg::getDureeEntre

`mg::getDureeEntre($_cmd_id, $_value, $_startDate, $_endDate, $_unit = 60, $_operator = '==')` Calcule pendant combien de temps une commande historisée a respecté une condition sur une période (par exemple : durée où la puissance a dépassé 10 W aujourd'hui).

- **Paramètres** : `$_cmd_id` : commande, par son id ou sa référence `#[Objet][Equipement][Commande]#` ; `$_value` : valeur de comparaison (nombre, virgule acceptée, ou texte) ; `$_startDate` / `$_endDate` : début et fin de période, dans tout format de date compris par PHP ou Jeedom (`'today'`, `'now'`, `'-1 day'`, tags acceptés) ; `$_unit` : unité du résultat en secondes, 1 = secondes, 60 = minutes, 3600 = heures (défaut 60) ; `$_operator` : `==`, `!=`, `>`, `>=`, `<`, `<=` (défaut `==`).
- **Retour** : la durée arrondie à l'unité inférieure ; `0` si la commande est introuvable ou non historisée, si l'unité ou une date est invalide, ou si la période commence dans le futur.
- **Exemple** : `$minutes = mg::getDureeEntre('#[Piscine][Pompe][Puissance]#', 10, 'today', 'now', 60, '>');`
- **Remarque** : une fin de période dans le futur est ramenée à maintenant ; si `$_value` n'est pas un nombre, la comparaison est forcément une égalité de texte, sans tenir compte de la casse.

mg::lastMvmt

`mg::lastMvmt($infNbMvmt = '', &$nbMvmt = 0)` Donne le nombre de minutes écoulées depuis le dernier mouvement sur un détecteur.

- **Paramètres** : `$infNbMvmt` : la commande info de mouvement à surveiller ; `$nbMvmt` : variable recevant la valeur courante de cette commande (nombre de mouvements en cours).
- **Retour** : la durée en minutes depuis le dernier changement de la commande ; 0 si un mouvement est en cours ou si la date est inconnue.
- **Exemple** : `$nb = 0; $minutes = mg::lastMvmt('#[Couloir][Détecteur][Mouvement]#', $nb);`

mg::gererMinuterie

`mg::gererMinuterie($equipEcl, $infNbMvmt, int $timer = 2, int $cdExtinction = 0, int $cdAllumage = 0)` Gère une lumière sur détection de mouvement : allume tant qu'il y a du mouvement, éteint après `$timer` minutes sans mouvement, et replanifie le scénario courant pour se réévaluer.

- **Paramètres** : `$equipEcl` : l'équipement lumière (`#[Couloir][Lampe]#`), qui doit porter une commande `Etat` et soit `Slider Intensité`, soit `On`/`Off` ; `$infNbMvmt` : la commande info de mouvement (nombre de mouvements en cours) ; `$timer` : délai d'extinction en minutes (défaut 2, minimum 1) ; `$cdExtinction` : si > 0, force l'extinction (défaut 0) ; `$cdAllumage` : si > 0, force l'allumage (défaut 0).
- **Retour** : aucun.
- **Exemple** : `mg::gererMinuterie('#[Couloir][Lampe]#', '#[Couloir][Détecteur][Mouvement]#', 5);`
- **Remarque** : programme à chaque appel le scénario courant dans `$timer` minutes. Allumage si mouvement ou `$cdAllumage`, sauf si `$cdExtinction` est demandé ; extinction si `$cdExtinction` ou plus de `$timer` minutes sans mouvement. Avec `Slider Intensité` la lampe est mise à 100 / 0, sinon `On` / `Off`.
- **Remarque** : aucune commande n'est envoyée si la lampe a changé d'état il y a moins de 5 secondes, ni si elle est déjà dans l'état visé ; environ une fois par heure, l'extinction est renvoyée même à une lampe déjà éteinte pour la garder communicante.

Retour au sommaire

---

5. Équipements

mg::getEquipement

`mg::getEquipement($cmd)` Donne l'équipement auquel appartient une commande.

- **Paramètres** : `$cmd` : commande, par sa référence ou son id (avec ou sans `#`).
- **Retour** : la référence de l'équipement `#[Objet][Equipement]#` ; `null` si la commande ou son équipement est introuvable.
- **Exemple** : `$equip = mg::getEquipement('#[Salon][Lampe][Etat]#');`

mg::setEquipement

`mg::setEquipement($eqLogic, $action)` Affiche, masque, active ou désactive un équipement.

- **Paramètres** : `$eqLogic` : équipement, par sa référence `#[Objet][Equipement]#` ou son id ; `$action` : `show`, `hide`, `activate` ou `deactivate` (casse indifférente).
- **Retour** : `true` si l'ordre est passé ; `false` si l'équipement est introuvable ou l'action inconnue.
- **Exemple** : `mg::setEquipement('#[Jardin][Arrosage]#', 'deactivate');`

mg::getConfigEquipement

`mg::getConfigEquipement($cmd, $configName)` Lit un paramètre de configuration de l'équipement auquel appartient une commande.

- **Paramètres** : `$cmd` : commande, par sa référence ou son id ; `$configName` : nom du paramètre de configuration de l'équipement.
- **Retour** : la valeur du paramètre ; `null` si la commande ou son équipement est introuvable.
- **Exemple** : `$ip = mg::getConfigEquipement('#[Entrée][Caméra][Etat]#', 'ip');`

mg::configurerEquipement

`mg::configurerEquipement($typeName, $equipement, $name, $newValue = '')` Lit ou modifie un paramètre de configuration interne d'un équipement d'un plugin donné.

- **Paramètres** : `$typeName` : nom technique du plugin (ex. `'virtual'`) ; `$equipement` : nom court de l'équipement (sans l'objet) ; `$name` : nom du paramètre de configuration ; `$newValue` : nouvelle valeur ; s'il est omis, la fonction lit seulement.
- **Retour** : en lecture, la valeur actuelle ; en écriture, rien ; `null` si le plugin est inconnu ou l'équipement introuvable.
- **Exemple** : `mg::configurerEquipement('rfxcom', 'Porte Garage', 'battery_warning_threshold', 55);`
- **Remarque** : seul le premier équipement portant ce nom est traité ; une valeur `0`, `'0'` ou vide passée explicitement est bien écrite ; l'équipement n'est sauvegardé que si la valeur change.

mg::estActif

`mg::estActif($eqLogic)` Indique si un équipement est activé.

- **Paramètres** : `$eqLogic` : équipement, par sa référence `#[Objet][Equipement]#` ou son id (`12`, `#12#`, `#eqLogic12#`).
- **Retour** : `true` si l'équipement est actif, `false` s'il est désactivé, introuvable ou si le paramètre est vide.
- **Exemple** : `if (mg::estActif('#[Salon][Lampe]#')) { ... }`

mg::getTypeEqui

`mg::getTypeEqui($eqLogic)` Donne le plugin (type) d'un équipement, par exemple `virtual`.

- **Paramètres** : `$eqLogic` : équipement, par sa référence `#[Objet][Equipement]#` ou son id (`12`, `#12#`, `#eqLogic12#`).
- **Retour** : le nom technique du plugin ; `false` si l'équipement est introuvable ou le paramètre vide.
- **Exemple** : `$type = mg::getTypeEqui('#[Salon][Lampe]#');`

mg::clonerEquipement

`mg::clonerEquipement($eqType, $idEquip_Source, $idEquip_Cible, $file_JSON = null, $onlyNonEmpty = false)` Copie les commandes d'un équipement vers un autre du même plugin (création des manquantes, mise à jour des existantes), avec export ou import possible via un fichier JSON.

- **Paramètres** : `$eqType` : type (plugin) des équipements, ex. `'mgThermostat'` ; `$idEquip_Source` : identifiant de l'équipement modèle (0 = lire le fichier JSON) ; `$idEquip_Cible` : identifiant de l'équipement à compléter (0 = export JSON seul) ; `$file_JSON` : chemin complet du fichier JSON écrit depuis la source ou lu à défaut de source (défaut `null`) ; `$onlyNonEmpty` : si `true`, seules les propriétés non vides sont recopiées (défaut `false`).
- **Retour** : aucun.
- **Exemple** : `mg::clonerEquipement('mgThermostat', 12, 34);`
- **Usage** : réservée aux plugins mgDomo (utilisée par mgThermostat).
- **Remarque** : aucune commande n'est supprimée ; une commande est reconnue par son identifiant logique ET son nom (sinon elle est créée). La valeur et le cache des commandes ne sont pas copiés. Lève une exception si source ou cible n'est pas du type `$eqType`, ou si aucune source ni fichier JSON lisible n'est disponible ; cible introuvable = arrêt silencieux.

Retour au sommaire

---

6. Variables et tags

mg::getVar

`mg::getVar($varName, $default = null)` Lit une variable de scénario Jeedom (celles de l'écran Variables), en conservant son type.

- **Paramètres** : `$varName` : nom exact de la variable (sans `#`, sans `variable(...)`) ; `$default` : valeur rendue si la variable n'existe pas (défaut `null`).
- **Retour** : la valeur de la variable (`0`, `'0'` et un tableau vide sont bien rendus) ; `$default` si la variable n'existe pas ou contient une chaîne vide ; `null` si le nom est vide.
- **Exemple** : `$mode = mg::getVar('ModeAbsence', 0);`
- **Exemple** : `$pos = mg::getVar('dernierePosition', [0, 0]);`
- **Remarque** : une variable enregistrée à `false` ou à `''` est lue comme absente et rend `$default` (fonctionnement natif de Jeedom) ; fonctionne aussi hors scénario (variables communes à tout Jeedom).

mg::setVar

`mg::setVar($varName, $value = "")` Crée ou modifie une variable de scénario Jeedom ; un tableau est accepté.

- **Paramètres** : `$varName` : nom de la variable ; `$value` : valeur à enregistrer, nombre, texte ou tableau (défaut chaîne vide).
- **Retour** : aucun.
- **Exemple** : `mg::setVar('ModeAbsence', 1);`
- **Exemple** : `mg::setVar('dernierePosition', [43.6, 7.1]);`
- **Remarque** : un nom contenant d'autres caractères que lettres sans accent, chiffres, `_` et `-` provoque un avertissement dans le log mais la variable est quand même enregistrée ; `false` est enregistré comme chaîne vide (relu comme absent par `mg::getVar`).

mg::unsetVar

`mg::unsetVar($varName)` Supprime une variable de scénario Jeedom.

- **Paramètres** : `$varName` : nom de la variable.
- **Retour** : aucun.
- **Exemple** : `mg::unsetVar('ModeAbsence');`
- **Remarque** : une variable inexistante est simplement signalée dans le log ; une variable de valeur vide est bien supprimée.

mg::getTag

`mg::getTag($tag = '', $default = '')` Lit la valeur d'un tag du scénario en cours (tag passé au lancement, `#trigger#`, tags système comme `#heure#`).

- **Paramètres** : `$tag` : nom du tag, avec ou sans les `#` ; `$default` : valeur rendue si le tag est vide, inconnu ou hors scénario (défaut chaîne vide).
- **Retour** : la valeur du tag, sans guillemets autour ; `$default` si elle est vide, si le tag n'existe pas ou si l'appel a lieu hors scénario ; `null` si le nom est vide.
- **Exemple** : `$mode = mg::getTag('mode', 'jour');`

mg::setTag

`mg::setTag($tag, $value = '')` Crée ou modifie un tag du scénario en cours ; le tag n'existe que pendant cette exécution.

- **Paramètres** : `$tag` : nom du tag, avec ou sans les `#` ; `$value` : valeur à donner (défaut chaîne vide).
- **Retour** : `true` si le tag est posé, `false` si le nom est vide ou en cas d'erreur.
- **Exemple** : `mg::setTag('piece', 'Salon');`

Retour au sommaire

---

7. Scénarios

mg::getScenario

`mg::getScenario($scenario_id, &$name = '')` Donne l'état d'un scénario et, au passage, son nom.

- **Paramètres** : `$scenario_id` : numéro (id) du scénario ; `&$name` : variable qui reçoit le nom du scénario (facultatif).
- **Retour** : `1` en cours, `0` arrêté, `-1` désactivé, `2` si le scénario n'existe pas.
- **Exemple** : `if (mg::getScenario(48, $nom) == 1) mg::message('', "$nom tourne encore");`

mg::setScenario

`mg::setScenario($scenario_id, $action, $tags = '')` Pilote un autre scénario : le lancer, l'arrêter, l'activer ou le désactiver, en lui passant éventuellement des tags.

- **Paramètres** : `$scenario_id` : numéro du scénario ; `$action` : `start`, `startsync` (lance et attend la fin), `stop`, `activate`, `deactivate` ou `resetRepeatIfStatus` (casse indifférente) ; `$tags` : tags transmis au lancement, en texte `'tag1=oui tag2=non'` ou en tableau `['tag 1' => 'oui']` (défaut aucun, utilisé seulement par `start` et `startsync`).
- **Retour** : pour `start` / `startsync`, le retour de l'action de lancement de Jeedom ; `true` pour les autres actions ; `false` si le numéro est invalide, le scénario introuvable ou l'action inconnue.
- **Exemple** : `mg::setScenario(48, 'startsync', 'mode=nuit piece=salon');`

mg::arreterScenario

`mg::arreterScenario()` Arrête immédiatement le scénario en cours, y compris le bloc code qui l'appelle.

- **Paramètres** : aucun.
- **Retour** : aucun ; la fonction interrompt l'exécution (les lignes qui suivent ne sont jamais exécutées).
- **Exemple** : `if (mg::getCmd('#[Maison][Alarme][Etat]#') == 0) mg::arreterScenario();`
- **Remarque** : l'arrêt est obtenu en levant une exception : ne pas placer l'appel dans un `try { } catch (Exception $e) { }` qui l'intercepterait.

mg::retournerScenario

`mg::retournerScenario($message)` Définit le texte ou la valeur renvoyée par le scénario, par exemple la réponse d'une interaction.

- **Paramètres** : `$message` : texte ou valeur à renvoyer (`'0'` est accepté).
- **Retour** : `true` si le retour est posé, `false` si le message est vide.
- **Exemple** : `mg::retournerScenario('Ok, je ferme les volets du salon');`
- **Remarque** : à appeler uniquement depuis un scénario (contexte scénario obligatoire).

mg::setCron

`mg::setCron($scenarioID = '', $cron = null)` Programme la prochaine exécution d'un scénario en remplaçant sa programmation, soit à une date précise, soit selon un masque cron.

- **Paramètres** : `$scenarioID` : numéro du scénario (défaut : le scénario appelant) ; `$cron` : horodatage (timestamp) de la date voulue, ou masque cron classique comme `'*/5 * * * *'`.
- **Retour** : aucun.
- **Exemple** : `mg::setCron('', time() + 15*60);`
- **Remarque** : un `$cron` vide est refusé (la programmation n'est pas effacée) ; si la programmation est déjà identique, rien n'est réécrit ; un horodatage est arrondi à la minute ; un numéro de scénario inexistant ne fait rien, avec une ERROR au log.

mg::convertirCronEnHeure

`mg::convertirCronEnHeure($cron = '')` Convertit une programmation ponctuelle datée (cron à 6 champs avec l'année) en timestamp.

- **Paramètres** : `$cron` : la programmation, ex. `'30 7 15 10 * 2026'` (défaut `''` = programmation du scénario courant).
- **Retour** : le timestamp correspondant, ou `false` si la programmation est récurrente (avec `*`), incomplète, ou absente hors scénario.
- **Exemple** : `$quand = mg::convertirCronEnHeure('30 7 15 10 * 2026');`
- **Remarque** : le champ jour de la semaine est ignoré ; échec signalé en ERROR au log.

mg::Declencheur

`mg::Declencheur($cmd = '', $part = '', $msg = 1)` Indique ce qui a déclenché le scénario : soit teste si le déclencheur correspond à un texte donné, soit renvoie une partie ou la totalité de son nom.

- **Paramètres** : `$cmd` : commande (`#[Objet][Equipement][Commande]#` ou `#id#`) ou simple morceau de texte à chercher dans le nom du déclencheur, recherche insensible à la casse et aux accents (défaut vide) ; `$part` : utilisé seulement si `$cmd` est vide, numéro du segment à renvoyer : 1 objet, 2 équipement, 3 commande ; `$msg` : 1 pour tracer le résultat dans le log, 0 pour rester silencieux (défaut 1).
- **Retour** : avec `$cmd` : `true` si le déclencheur correspond, `false` sinon ; avec `$part` : le segment demandé, ou chaîne vide s'il n'existe pas ; sans rien : le nom complet du déclencheur.
- **Exemple** : `if (mg::Declencheur('Porte Entrée')) { ... }` ou `$piece = mg::Declencheur('', 1);`
- **Remarque** : la recherche par morceau de texte peut matcher plus large que prévu (`'salon'` correspond aussi à l'objet `[Salon]`) : choisir un texte discriminant ; hors scénario,

renvoie `false` (avec `$cmd`) ou une chaîne vide.

mg::wait

`mg::wait($condition, $timeout = 7200)` Met le scénario en pause jusqu'à ce qu'une condition soit vraie, ou jusqu'à l'expiration d'un délai.

- **Paramètres** : `$condition` : condition écrite comme dans le testeur d'expression Jeedom (ex. `'#[Salon][Détecteur][Présence]# == 0'`) ; `$timeout` : délai maximal en secondes, borné entre 1 et 7200 (défaut 7200).
- **Retour** : `true` si la condition est vraie à la sortie de l'attente, `false` si le délai est dépassé ou en cas d'erreur.
- **Exemple** : `if (!mg::wait('#[Entrée][Porte][Etat]# == 0', 120)) mg::message('', 'Porte toujours ouverte');`
- **Remarque** : le délai réel est aussi limité par le temps d'exécution maximal des scénarios réglé dans Jeedom ; une condition vide est refusée : ERROR au log et `false` immédiatement, sans attente.

mg::setAttenteCmd

`mg::setAttenteCmd($wait=true)` Choisit si les commandes lancées ensuite par `mg::setCmd` attendent leur réponse ou s'enchaînent sans attendre.

- **Paramètres** : `$wait` : `true` pour attendre la réponse (fonctionnement normal), `false` pour enchaîner sans attendre (défaut `true`).
- **Retour** : aucun.
- **Exemple** : `mg::setAttenteCmd(false); mg::setCmd('#[Salon][TTS][Parle]#', '', 'Bonjour'); mg::setAttenteCmd(true);`
- **Remarque** : utile pour une commande qui fait une requête vers un appareil lent ou absent, qui bloquerait sinon le scénario jusqu'au délai d'attente ; la valeur doit être un vrai booléen (`true` / `false`), sinon elle est ignorée.

mg::evaluerCondition

`mg::evaluerCondition($condition)` Évalue une condition écrite comme dans un scénario Jeedom (commandes `#[Objet][Équipement][Commande]#`, `variable(nom)`, opérateurs) et dit si elle est vraie.

- **Paramètres** : `$condition` : la condition à évaluer, au format d'une condition de scénario.
- **Retour** : la chaîne `'1'` si la condition est vraie, `'0'` sinon (des chaînes, pas des booléens).
- **Exemple** : `if (mg::evaluerCondition("#[Salon][Lampe][Etat]# == 1 && variable(Mode) == 'Nuit'") == '1') { ... }`
- **Remarque** : le détail de l'évaluation (condition lisible, expression calculée, résultat) est écrit en INFO dans le log du scénario.

Retour au sommaire

---

8. Réglages, tables et utilisateurs

mg::getParam

`mg::getParam($section, $name, $valDefaut = null, $remarque = null)` Lit un réglage partagé de l'écosystème mgDomo dans la table `_tabParams` (section + nom). Si le réglage n'existe pas encore et qu'une valeur par défaut ET une remarque sont fournies, la ligne est créée automatiquement.

- **Paramètres** : `$section` : nom de la section (converti en minuscules) ; `$name` : nom du réglage (sensible à la casse) ; `$valDefaut` : valeur à utiliser si le réglage est absent (défaut `null` = aucune) ; `$remarque` : texte d'explication du réglage (défaut `null`).
- **Retour** : la valeur enregistrée (espaces de bord retirés) ; si le réglage est absent : `$valDefaut` ; si absent sans valeur par défaut, ou section/nom vide : chaîne vide `''`.
- **Exemple** : `$seuil = mg::getParam('Chauffage', 'seuilHorsGel', 7, 'Température de mise hors gel (°C)');`
- **Remarque** : une valeur déjà enregistrée n'est jamais écrasée ; la remarque, elle, est réalignée sur celle du code à chaque appel. Absent avec défaut mais sans remarque : WARNING, rien d'enregistré. Absent sans défaut : ERROR au log. Une valeur par défaut écrite sous forme de référence `#[Objet][Équipement][Commande]#` est stockée en identifiant. La valeur des réglages sensibles (clés, jetons, mots de passe) est masquée dans le log.
- **Remarque** : une valeur égale au marqueur « à configurer » du socle déclenche un message Jeedom invitant à renseigner le réglage dans la configuration mgDomo.

mg::getValSql

`mg::getValSql($nomTab, $key0 = '', $key1 = '', $name = '', $defaut = '')` Lit une valeur dans une table SQL de configuration mgDomo, en désignant la ligne par sa clé primaire et la colonne par son nom.

- **Paramètres** : `$nomTab` : nom de la table ; `$key0` : valeur du premier élément de la clé primaire (obligatoire, 0 accepté) ; `$key1` : valeur du second élément de la clé, pour les tables à clé double (défaut '') ; `$name` : nom de la colonne à lire (obligatoire) ; `$defaut` : valeur rendue si la ligne est introuvable ou en cas d'erreur (défaut '').
- **Retour** : la valeur lue, en texte et sans espaces superflus ; `$defaut` si la table, la ligne ou la colonne est invalide ou introuvable.
- **Exemple** : `$ip = mg::getValSql('_tabUsers', 'JC', '', 'IP', '');`
- **Remarque** : si plusieurs lignes correspondent (second élément de clé non fourni sur une table à clé double), la première valeur est rendue avec un avertissement au journal.

mg::getTableSql

`mg::getTableSql($nomTab, $order = '', $sens = '')` Lit toute une table SQL et la rend sous forme de tableau indexé par sa clé primaire (une ou deux colonnes).

- **Paramètres** : `$nomTab` : nom de la table ; `$order` : colonne de tri (défaut '' = colonnes de la clé primaire) ; `$sens` : `ASC` ou `DESC` (défaut '' = croissant).

- **Retour** : tableau `[clé][colonne] => valeur` pour une clé simple, `[clé1][clé2][colonne] => valeur` pour une clé double (les colonnes de clé ne sont pas répétées dans les lignes) ; une table sans clé primaire est indexée par numéro de ligne ; tableau vide si un nom est invalide ou si la table est absente.
- **Exemple** : `$users = mg::getTableSql('_tabUsers'); foreach ($users as $code => $u) mg::message('', "$code : " . $u['IP']);`
- **Remarque** : les noms de table et de colonne n'acceptent que lettres, chiffres et soulignés. Une table absente ou une requête en échec donne une ERROR au log et un tableau vide.

mg::assurerDonneesInitiales

`mg::assurerDonneesInitiales($nomTab, $lignes, $clesUniques = null)` Remplit une table de configuration avec des lignes de référence sans jamais écraser les données existantes : soit uniquement si la table est vide, soit ligne par ligne pour ajouter seulement les lignes manquantes.

- **Paramètres** : `$nomTab` : nom de la table ; `$lignes` : liste de lignes, chacune un tableau `colonne => valeur` ; `$clesUniques` : liste des colonnes qui identifient une ligne (défaut `null` = n'insérer que si la table est entièrement vide ; fourni = n'insérer que les lignes absentes sur ces colonnes).
- **Retour** : nombre de lignes insérées (0 si rien à faire), ou `false` en cas d'erreur (table inconnue, liste vide, colonne de clé inconnue, échec SQL avec annulation de toutes les insertions).
- **Exemple** : `mg::assurerDonneesInitiales('_tabComplements', array(array('nom' => 'Exemple', 'valeur' => '1')), array('nom'));`
- **Remarque** : les colonnes non fournies reçoivent 0 (colonnes numériques) ou '' (texte). Les lignes déjà présentes ne sont ni relues ni modifiées. En mode ligne par ligne, une ligne de référence sans valeur pour une des colonnes de `$clesUniques` est refusée (ERROR au log).
- **Usage** : réservée aux plugins mgDomo (amorçage des tables à l'installation ou à la mise à jour).

mg::getConfigJeedom

`mg::getConfigJeedom($plugin = 'core', $key = 'api')` Lit une valeur de configuration de Jeedom (du cœur par défaut, ou d'un plugin), déchiffrée si besoin.

- **Paramètres** : `$plugin` : `'core'` ou l'identifiant d'un plugin (défaut `'core'`) ; `$key` : la clé de configuration (défaut `'api'`, par exemple `'internalAddr'` pour l'adresse interne).
- **Retour** : la valeur lue (chaîne ou tableau), ou `null` si la clé est vide ou absente.
- **Exemple** : `$adresse = mg::getConfigJeedom('core', 'internalAddr');`
- **Remarque** : la valeur elle-même n'est jamais écrite dans le log (seule sa longueur l'est).

mg::getPhotoUtilisateur

`mg::getPhotoUtilisateur($code, $defaut = '')` Donne la photo d'un utilisateur de l'annuaire mgDomo (fiche utilisateur de mgDomo).

- **Paramètres** : `$code` : code de l'utilisateur dans l'annuaire ; `$defaut` : valeur rendue si aucune photo (défaut `''`).
- **Retour** : le chemin de la photo relatif au serveur (ex. `plugins/mgDomo/data/users/XX.png`), ou `$defaut` si le code est vide, inconnu ou sans photo.
- **Exemple** : `$photo = mg::getPhotoUtilisateur('AB', 'plugins/mgDomo/data/users/inconnu.png');`
- **Remarque** : un code inconnu laisse un WARNING au log.

mg::getCouleurUtilisateur

`mg::getCouleurUtilisateur($code, $defaut = 'steelblue')` Donne la couleur attribuée à un utilisateur de l'annuaire mgDomo (utilisée par les cartes, graphiques et widgets).

- **Paramètres** : `$code` : code de l'utilisateur dans l'annuaire ; `$defaut` : couleur rendue si aucune n'est définie (défaut `'steelblue'`).
- **Retour** : une couleur CSS (nom ou `#rrggbb`), ou `$defaut` si le code est vide, inconnu ou sans couleur.
- **Exemple** : `$couleur = mg::getCouleurUtilisateur('AB');`
- **Remarque** : un code inconnu laisse un WARNING au log.

mg::getCleApiDomo

`mg::getCleApiDomo($_eqLogicIds = array())` Fournit la clé API d'un compte Jeedom dédié et restreint (`_mgDomo`), à utiliser dans les pages publiques des plugins mgDomo à la place de la clé du cœur ; ce compte n'a accès qu'aux équipements explicitement autorisés.

- **Paramètres** : `$_eqLogicIds` : tableau d'identifiants d'équipements à autoriser en lecture et exécution (défaut : aucun).
- **Retour** : la clé API du compte dédié, ou `''` en cas d'échec.
- **Exemple** : `$cle = mg::getCleApiDomo(array($eqLogic->getId()));`
- **Usage** : réservée aux plugins mgDomo (pages web, cartes, tableaux de bord publics).
- **Remarque** : crée le compte s'il n'existe pas, le force en profil restreint et actif ; les droits s'additionnent d'un appel à l'autre, aucun n'est retiré.

Retour au sommaire

---

9. Dates, durées et calculs

mg::formaterDuree

`mg::formaterDuree($secondes, $mask = 'j:hh:mm:ss')` Met en forme une durée exprimée en secondes selon un masque : `j` jours, `h` heures, `m` minutes, `s` secondes ; une lettre = sans zéro devant, deux lettres = sur deux chiffres, trois lettres = forme longue avec unité (`5j`, `3h`, `12mn`, `27sec`).

- **Paramètres** : `$secondes` : durée en secondes (arrondie ; une durée négative est prise en valeur absolue, une valeur vide vaut 0) ; `$mask` : masque de mise en forme (défaut `j:hh:mm:ss`).
- **Retour** : la durée en texte.

- **Exemple** : `$txt = mg::formaterDuree(90061);` (donne `1j 01:01:01`)
- **Exemple** : `$txt = mg::formaterDuree(327, 'mm:ss');` (donne `05:27`)
- **Remarque** : les jours sont toujours suivis de « j » et le « : » qui suit devient une espace. Avec `j` ou `jjj` et zéro jour, la partie jours disparaît ; avec `jj` et zéro jour, elle s'affiche `0j`. Si le masque ne contient pas de `j`, les jours sont comptés dans les heures (90061 s en `hh:mm:ss` donne `25:01:01`).

mg::TimeBetween

`mg::TimeBetween($start, $time, $end)` Indique si une heure se situe dans une plage horaire, sans tenir compte du jour, y compris pour une plage qui passe minuit (ex. 22:00 à 06:00).

- **Paramètres** : `$start` : début de plage ; `$time` : heure à tester ; `$end` : fin de plage ; chacun en timestamp ou en texte `'hh:mm'`.
- **Retour** : l'entier `1` si l'heure est dans la plage (début inclus, fin exclue), sinon `0` ; une plage dont début et fin sont sur la même minute rend toujours `0`.
- **Exemple** : `if (mg::TimeBetween('22:00', time(), '06:30')) { ... }`
- **Remarque** : attention à l'ordre des paramètres (début, heure, fin). Comparaison à la minute. Un nombre est toujours lu comme un timestamp ; un texte illisible déclenche un WARNING et vaut minuit (heure de l'époque Unix).

mg::getSoleil

`mg::getSoleil($timestamp, $latitude, $longitude, $angleSoir = null)` Donne la position du soleil à un instant donné et les heures de lever, coucher et crépuscule du jour correspondant.

- **Paramètres** : `$timestamp` : l'instant voulu (timestamp) ; `$latitude` : latitude en degrés (négative au sud) ; `$longitude` : longitude en degrés (négative à l'ouest) ; `$angleSoir` : facultatif, hauteur du soleil sous l'horizon visée le soir, en degrés (ex. `-7`) (défaut `null` = pas de clé `soir`).
- **Retour** : tableau avec `azimuth` (degrés, compté depuis le nord dans le sens horaire), `altitude` (hauteur au-dessus de l'horizon en degrés, négative la nuit), et en timestamps `sunrise`, `sunriseEnd`, `sunset`, `dusk` (début du crépuscule nautique, soleil à -6°), `night` (nuit astronomique) ; avec `$angleSoir`, en plus `soir` : l'heure où le soleil descend à cet angle (`soir` = `dusk` pour -6).
- **Exemple** : `$sol = mg::getSoleil(time(), $lat, $lon); if ($sol['altitude'] < -6) { ... }`
- **Exemple** : `$nuit = mg::getSoleil(time(), $lat, $lon, -7)['soir'];`
- **Remarque** : les heures sont celles du jour du timestamp (calculées pour midi) ; elles valent `null` quand l'événement n'a pas lieu (régions polaires).

mg::getDistance

`mg::getDistance($lat1, $lng1, $lat2, $lng2, $unit = 'm', &$azimut = 0)` Calcule la distance à vol d'oiseau entre deux points GPS, et en option la direction du second vu depuis le premier.

- **Paramètres** : `$lat1`, `$lng1` : coordonnées du point de départ (degrés) ; `$lat2`, `$lng2` : coordonnées du point d'arrivée (degrés) ; `$unit` : `'m'` pour des mètres, `'k'` pour des

kilomètres (défaut `'m'`) ; `$azimut` : variable recevant le cap du point 2 vu du point 1, en degrés de 0 à 360 (0 = nord, 90 = est).
- **Retour** : la distance (arrondie au mètre, puis divisée par 1000 si `'k'`).
- **Exemple** : `$az = 0; $d = mg::getDistance($latMaison, $lonMaison, $latTel, $lonTel, 'k', $az);`
- **Remarque** : une coordonnée vide compte pour 0. Le calcul utilise le rayon équatorial terrestre, ce qui majore légèrement la distance (quelques pour mille).

mg::getAltitude

`mg::getAltitude($latitude, $longitude, $nbEssais = 1)` Donne l'altitude d'un point GPS grâce au service d'altimétrie de l'IGN (précision de l'ordre du mètre).

- **Paramètres** : `$latitude` : latitude en degrés ; `$longitude` : longitude en degrés ; `$nbEssais` : nombre de tentatives en cas d'échec réseau (défaut 1).
- **Retour** : l'altitude en mètres, ou `-999` si les coordonnées sont invalides, hors zone couverte, ou en cas d'échec.
- **Exemple** : `$alt = mg::getAltitude($lat, $lon, 2);`
- **Remarque** : nécessite internet ; couverture : métropole et DROM (pas la Nouvelle-Calédonie ni l'étranger). Chaque appel attend un quart de seconde : pour plusieurs points, utiliser `mg::getAltitudeBatch`.

mg::getAltitudeBatch

`mg::getAltitudeBatch(array $points, int $nbEssais = 2)` Donne les altitudes de tout un lot de points GPS en une seule requête au service IGN, beaucoup plus rapide que des appels un par un.

- **Paramètres** : `$points` : tableau de points `[[lat, lon], [lat, lon], ...]` (clés libres) ; `$nbEssais` : nombre de tentatives par paquet en cas d'échec (défaut 2).
- **Retour** : tableau des altitudes en mètres, avec les MÊMES clés que l'entrée ; `-999` pour un point invalide, hors zone ou dont la requête a échoué ; tableau vide si l'entrée est vide.
- **Exemple** : `$alts = mg::getAltitudeBatch(array('depart' => [45.1, 5.7], 'arrivee' => [45.2, 5.8]));`
- **Remarque** : nécessite internet ; les lots de plus de 5000 points sont découpés automatiquement ; un paquet en échec est signalé en ERROR au log.

mg::trierTableau

`mg::trierTableau($array, $key, $ordre=SORT_ASC)` Trie un tableau de lignes selon une de leurs clés, en conservant les clés d'origine des lignes.

- **Paramètres** : `$array` : tableau à trier (tableau de tableaux, ou tableau de valeurs simples triées sur elles-mêmes) ; `$key` : clé de tri (nom ou numéro de colonne) ; `$ordre` : `SORT_ASC` ou `SORT_DESC` (défaut `SORT_ASC`).
- **Retour** : le tableau trié, clés d'origine préservées ; les lignes qui n'ont pas la clé de tri sont gardées et placées à la fin ; tableau vide si l'entrée est vide.
- **Exemple** : `$tri = mg::trierTableau($releves, 'temperature', SORT_DESC);`

- **Remarque** : les clés n'étant pas renumérotées, utiliser `array_values()` pour accéder au premier élément trié par `[0]`.

mg::getICS

`mg::getICS($url, $start = 0, $end = 0)` Lit un agenda au format ICS (fichier local ou adresse web) et renvoie les événements d'une période, récurrences comprises, triés par date de début.

- **Paramètres** : `$url` : chemin ou adresse `http(s)://` de l'agenda ; `$start` : jour de début (timestamp, défaut 0 = aujourd'hui), pris à 00:00:00 ; `$end` : jour de fin (timestamp, défaut 0 = aujourd'hui), pris à 23:59:59.
- **Retour** : tableau indexé 0..n d'événements, chacun avec les clés `start` et `end` (format `Y-m-d H:i:s`), `title`, `description`, `location` ; tableau vide si aucun événement ou en cas d'erreur.
- **Exemple** : `$evts = mg::getICS('https://exemple.org/agenda.ics', time(), time() + 7*86400);`
- **Remarque** : adresse vide, agenda injoignable (délai 30 s) ou illisible : ERROR au log et tableau vide. Un événement sans fin a sa fin égale à son début.

Retour au sommaire

---

10. Voix, médias, mails et réseau

mg::VoiceRSS

`mg::VoiceRSS($nom_File_TTS, $message, $pathMedia = null)` Transforme un texte en fichier MP3 parlé par le service en ligne VoiceRSS, et l'enregistre dans le dossier des médias des enceintes (racine du partage « musicshare »), prêt à être joué par une enceinte Sonos.

- **Paramètres** : `$nom_File_TTS` : nom du fichier MP3 à créer dans le dossier des médias ; `$message` : texte à prononcer ; `$pathMedia` : ignoré, conservé pour compatibilité (défaut `null`).
- **Retour** : `true` si le fichier est écrit, `false` sinon (clé non configurée, dossier des médias absent, erreur du service, écriture impossible) ; le dossier est vérifié avant l'appel au service.
- **Exemple** : `mg::VoiceRSS('VoiceRSS.mp3', 'Bonjour, il fait 18 degrés dehors');`
- **Remarque** : nécessite une clé VoiceRSS dans les paramètres (section `Media`, `API_VoiceRSS`) ; la voix se règle par `Media` / `TTS_Voix` (défaut `Bette`, voix féminine ; `Axel` = voix masculine). Un fichier existant du même nom est écrasé. Nécessite internet.

mg::getEtatMedia

`mg::getEtatMedia()` Donne l'état du dossier des médias des enceintes (jingles, fichiers de synthèse vocale) et de son partage réseau « musicshare ». Utilisée par le contrôle de santé de mgDomo.

- **Paramètres** : aucun.

- **Retour** : tableau `['dossier' => chemin absolu du dossier des médias, 'partage' => chemin déclaré par le partage « musicshare » ('' s'il n'existe pas), 'smbd' => true si le service de partage Samba est actif]`.
- **Exemple** : `$etat = mg::getEtatMedia(); if ($etat['partage'] == '') mg::message('', 'Partage des médias absent');`
- **Usage** : réservée aux plugins mgDomo (contrôle de santé de mgDomo).

mg::assurerMedia

`mg::assurerMedia($verifSeule = false)` Rend opérationnels le dossier des médias des enceintes et son partage réseau « musicshare » : création du dossier et de son sous-dossier `tts`, copie des jingles livrés manquants, et si besoin installation de Samba et création du partage. Peut être appelée plusieurs fois sans risque.

- **Paramètres** : `$verifSeule` : `true` = ne crée que le dossier et copie les jingles, sans aucune action système (pas d'installation ni de modification de la configuration Samba) (défaut `false`).
- **Retour** : `true` si le dossier et le partage sont opérationnels, `false` sinon (le motif est écrit dans le journal mgDomo).
- **Exemple** : `mg::assurerMedia(true);`
- **Remarque** : si un partage « musicshare » existe déjà, son dossier est adopté comme dossier des médias (paramètre `Media` / `PathMedia`) et le partage n'est jamais modifié. Sinon, la création du partage sauvegarde d'abord la configuration Samba, la valide, active le service et laisse un message au centre de messages Jeedom (à faire ensuite : indexer le partage dans Sonos et créer les favoris « VoiceRSS.mp3 » et « jingle_01.mp3 »).
- **Usage** : réservée aux plugins mgDomo (installation de mgDomo et préparation des annonces).

mg::securiserPartageMedia

`mg::securiserPartageMedia($smbConf = '/etc/samba/smb.conf', $simuler = false)` Protège le partage « musicshare » créé par mgDomo en y interdisant les fichiers cachés et les noms de fichiers PHP, afin qu'un invité du réseau ne puisse rien y déposer d'exécutable par le serveur web. Un partage créé par l'utilisateur n'est jamais touché.

- **Paramètres** : `$smbConf` : chemin du fichier de configuration Samba (défaut `/etc/samba/smb.conf`) ; `$simuler` : `true` = n'écrit rien et rend le contenu qui serait écrit (défaut `false`).
- **Retour** : `true` si rien n'est à faire ou si la protection a été posée, `false` en cas d'échec d'écriture ; en simulation, le texte complet de la configuration qui serait écrite.
- **Exemple** : `$apercu = mg::securiserPartageMedia('/etc/samba/smb.conf', true);`
- **Remarque** : sans fichier de configuration lisible, sans partage « musicshare », ou si l'administrateur a déjà sa propre règle d'interdiction, la fonction ne fait rien et rend `true`. Après un échec, aucun nouvel essai n'a lieu avant 24 heures.
- **Usage** : réservée aux plugins mgDomo (appelée automatiquement par la tâche de fond de mgDomo).

mg::envoyerMailPieceJointe

`mg::envoyerMailPieceJointe($user = '', $title = 'mail avec PJ', $file = null)` Envoie un mail avec un fichier joint par le plugin Mail de Jeedom.

- **Paramètres** : `$user` : nom de la commande d'envoi du destinataire sur l'équipement Mail configuré (obligatoire : sans destinataire, rien n'est envoyé) ; `$title` : sujet du mail (défaut `'mail avec PJ'`) ; `$file` : chemin complet du fichier à joindre (défaut `null`).
- **Retour** : aucun.
- **Exemple** : `mg::envoyerMailPieceJointe('Admin', 'Rapport du jour', '/tmp/rapport.pdf');`
- **Remarque** : prérequis : réglage `equip / equipPluginMail` renseigné dans la configuration mgDomo et une commande portant le nom `$user` sur cet équipement ; sinon rien n'est envoyé (message « à configurer » ou ERROR au log). Le corps du mail est « Envoi du fichier » suivi du chemin. La taille du fichier n'est pas contrôlée.

mg::WakeOnLan

`mg::WakeOnLan($ip, $mac)` Réveille un ordinateur du réseau local par un paquet « Wake-on-LAN », envoyé à l'adresse de diffusion de son sous-réseau.

- **Paramètres** : `$ip` : adresse IP de la machine (sert à calculer l'adresse de diffusion, en remplaçant le dernier nombre par 255) ; `$mac` : adresse MAC de la machine.
- **Retour** : aucun.
- **Exemple** : `mg::WakeOnLan('192.168.1.20', 'AA:BB:CC:DD:EE:FF');`
- **Remarque** : nécessite le programme `wakeonlan` sur le serveur Jeedom (`apt-get install wakeonlan`) ; le paquet part sur le port 7. Si l'IP ou la MAC est vide, rien n'est envoyé ; une IP illisible est signalée en erreur.

mg::estInternetCoupe

`mg::estInternetCoupe()` Indique si une coupure de la connexion internet est actuellement confirmée par la surveillance internet de mgDomo. Sert à éviter des envois qui échoueraient (mails, synthèse vocale en ligne, appels à une IA).

- **Paramètres** : aucun.
- **Retour** : `true` si une coupure est confirmée, `false` sinon (pas de coupure, surveillance non configurée, ou commande mémo inutilisable).
- **Exemple** : `if (!mg::estInternetCoupe()) mg::message('mail:moi', 'Rapport du jour');`
- **Remarque** : lit la commande mémo réglée dans les paramètres (section `Cmd`, `cmdAlarmeInternet`) ; si elle est supprimée ou n'est pas une commande info, la fonction répond `false` et écrit une erreur au plus une fois par heure dans le journal mgDomo.

mg::interrogerOpenAI

`mg::interrogerOpenAI($question, $temperature = 0.7)` Pose une question au modèle d'IA d'OpenAI (gpt-4o-mini) et rend sa réponse en texte.

- **Paramètres** : `$question` : texte de la question ; `$temperature` : créativité de la réponse, de 0 (précise) à 1 (variée) (défaut 0.7).

- **Retour** : le texte de la réponse ; en cas de problème, un texte d'erreur commençant par `openAI -` (clé non configurée, erreur réseau, erreur du service, réponse vide).
- **Exemple** : `$rep = mg::interrogerOpenAI('Donne une idée de repas pour ce soir en une phrase', 0.9);`
- **Remarque** : nécessite une clé dans les paramètres (section `IA`, `openai_key`) et internet ; délai maximum 30 secondes. La fonction ne lève pas d'exception : tester si la réponse commence par `openAI -` avant de l'utiliser.

mg::interrogerAnthropic

`mg::interrogerAnthropic($messages, $temperature = 0.7)` Pose une question au modèle d'IA Claude d'Anthropic (Claude Haiku) et rend sa réponse en texte.

- **Paramètres** : `$messages` : texte de la question ; `$temperature` : créativité de la réponse, de 0 (précise) à 1 (variée) (défaut 0.7).
- **Retour** : le texte de la réponse, sans espaces superflus ; si le service répond sans texte : `anthropic - ATTENTION : AUCUNE REPONSE TROUVEE`.
- **Exemple** : `try { $rep = mg::interrogerAnthropic('Reformule en une phrase : la porte du garage est ouverte'); } catch (Exception $e) { $rep = 'La porte du garage est ouverte'; }`
- **Remarque** : lève une exception si la clé n'est pas configurée (section `IA`, `anthropic_key`), en cas d'erreur réseau ou d'erreur du service : à encadrer par `try` / `catch` dans un scénario. Nécessite internet ; délai maximum 30 secondes ; réponse limitée à environ 4 000 jetons.

mg::convoyeurApk

`mg::convoyeurApk($canal, $message = '', $destinataire = '', $volume = 0, $sujet = '')` Fait parler, notifier ou régler un téléphone équipé de l'application mgDomo, ou lui fait envoyer un SMS ou un mail. L'ordre est déposé pour l'application et le téléphone est réveillé par une notification push : réaction en une seconde environ, sans jamais bloquer le scénario.

- **Paramètres** : `$canal` : `TTS` (lecture vocale), `NOTIF` (notification), `SMS`, `MAIL`, ou les fonctions propres `VOLUME` et `READ_VOL` (casse indifférente) ; `$message` : texte à dire, notifier ou envoyer (pour `VOLUME` : le niveau 0-100) ; `$destinataire` : pour `TTS`, `NOTIF`, `VOLUME`, `READ_VOL` le code utilisateur du téléphone visé ('' = téléphone passerelle) ; pour `SMS` le numéro, pour `MAIL` l'adresse (l'envoi part alors toujours du téléphone passerelle) ; `$volume` : volume média 0-100 appliqué avant une lecture `TTS` (défaut 0 = volume inchangé) ; `$sujet` : sujet du mail, ou titre de la notification pour `NOTIF`.
- **Retour** : `true` si l'ordre est émis (ce n'est pas une preuve de réception), `false` sinon ; pour `READ_VOL` : le volume média courant remonté par le téléphone (entier 0-100, 0 si inconnu).
- **Exemple** : `mg::convoyeurApk('TTS', 'La porte du garage est ouverte', 'JC', 60);`
- **Remarque** : le téléphone passerelle se règle dans les paramètres (section `equip`, `passerelle_APK`) ; sans lui, ou sans fiche utilisateur avec l'option « Téléphone connecté », rien n'est envoyé et une erreur est journalisée. Le canal `PLAY` n'est pas servi par ce convoyeur.
- **Remarque** : en `SMS`, le texte est ramené à l'alphabet SMS standard (accents rares translittérés, emojis retirés) ; un texte devenu vide n'est pas envoyé. En `MAIL`, la fiche

du téléphone passerelle doit porter l'option « Passerelle mail » ; le mail part du compte Google connecté au téléphone.
- **Usage** : réservée aux plugins mgDomo et à `mg::message()` pour les envois ; en scénario, préférer `mg::message()` sauf pour `VOLUME` et `READ_VOL`.

Retour au sommaire

---

11. Système et dépannage

mg::nettoyerRepertoire

`mg::nettoyerRepertoire($masque, $volMax = 5, $nbJoursMax = 31, $nbFiles = 999)` Fait le ménage dans un dossier : supprime les fichiers trop anciens, puis les plus anciens au-delà d'un volume total ou d'un nombre de fichiers.

- **Paramètres** : `$masque` : masque complet des fichiers, dossier compris (ex. `'/var/www/html/data/snapshots/*.jpg'`) ; `$volMax` : volume maximal conservé en Mo (défaut 5) ; `$nbJoursMax` : ancienneté maximale en jours (défaut 31) ; `$nbFiles` : nombre maximal de fichiers conservés (défaut 999).
- **Retour** : aucun.
- **Exemple** : `mg::nettoyerRepertoire('/var/www/html/data/snapshots/*.jpg', 50, 15, 500);`
- **Remarque** : suppression définitive ; les fichiers les plus récents sont gardés en priorité. Chaque suppression et le bilan sont tracés au log ; un échec de suppression (droits) donne une ERROR.

mg::eteindreJeedom

`mg::eteindreJeedom()` Demande l'arrêt complet de la machine Jeedom.

- **Paramètres** : aucun.
- **Retour** : aucun.
- **Exemple** : `mg::eteindreJeedom();`
- **Remarque** : la machine s'éteint réellement ; elle devra être rallumée manuellement.

mg::masquerCles

`mg::masquerCles($v)` Prépare un tableau (ou un texte) à sortir de l'installation : masque les valeurs dont le nom évoque un secret ou une identité, et anonymise le reste.

- **Paramètres** : `$v` : tableau (traité en profondeur) ou valeur simple.
- **Retour** : la même structure, avec `'***'` à la place des valeurs sensibles (mot de passe, clé, jeton, identifiant, téléphone, mail, hôte, numéro de série…), les adresses IP, MAC, mails, téléphones et coordonnées GPS masqués dans les textes ; les clés commençant par `comment` sont retirées.
- **Exemple** : `$export = mg::masquerCles($eqLogic->export(true));`
- **Usage** : réservée aux plugins mgDomo (diagnostics et rapports d'assistance).

mg::getRapportDepannage

`mg::getRapportDepannage($pluginId = '', $format = 'texte')` Produit un rapport texte anonymisé de l'installation (machine, santé Jeedom, plugins mgDomo, dernières erreurs d'un plugin), prêt à coller sur un forum.

- **Paramètres** : `$pluginId` : plugin dont on veut les dernières erreurs (défaut `''` = aucun) ; `$format` : `'texte'` ou `'markdown'` (rapport encadré en bloc de code) (défaut `'texte'`).
- **Retour** : le rapport sous forme de chaîne.
- **Exemple** : `$rapport = mg::getRapportDepannage('mgGeofence', 'markdown');`
- **Usage** : réservée aux plugins mgDomo (onglet Dépannage de mgDomo et diagnostics).
- **Remarque** : n'inclut que les points de santé en défaut et au plus les 15 dernières lignes d'erreur parmi les 100 dernières lignes du log du plugin ; adresses IP, mails, clés et jetons y sont masqués.

mg::creerPaquetDiagnostic

`mg::creerPaquetDiagnostic($pluginId = '', $avecJournauxTel = false)` Fabrique un fichier zip de diagnostic anonymisé de l'installation, à transmettre pour une demande d'aide.

- **Paramètres** : `$pluginId` : plugin concerné (défaut `''` = tout l'écosystème mgDomo) ; `$avecJournauxTel` : joindre aussi les journaux des téléphones, qui contiennent positions et SMS (défaut `false`).
- **Retour** : tableau `chemin` (fichier zip dans le dossier temporaire du système), `taille` (octets), `fichiers` (nombre d'éléments du zip).
- **Exemple** : `$paquet = mg::creerPaquetDiagnostic('mgVolets');`
- **Usage** : réservée aux plugins mgDomo (onglet Téléintervention de mgDomo).
- **Remarque** : contient le rapport de dépannage, les logs mgDomo et utiles du cœur, les équipements mgDomo, les réglages `_tabParams` hors sections secrètes et la liste des plugins mgDomo, le tout anonymisé. Le zip n'est pas supprimé : à l'appelant de le faire. Lève une exception si l'extension zip de PHP manque ou si le fichier ne peut être créé.

mg::getNomPlugin

`mg::getNomPlugin($pluginName = '')` Enregistre le nom du plugin qui utilise la bibliothèque.

- **Paramètres** : `$pluginName` : nom du plugin (défaut chaîne vide).
- **Retour** : aucun.
- **Exemple** : `mg::getNomPlugin('monPlugin');`
- **Usage** : réservée aux plugins mgDomo (à appeler au démarrage de la classe du plugin qui utilise la bibliothèque).

Retour au sommaire